

**A COMPARATIVE ANALYSIS ON CLASS IMBALANCE
TECHNIQUES IN NETWORK TRAFFIC CLASSIFICATION**

SHOVON DATTA RONY

**FACULTY OF COMPUTER SCIENCE AND INFORMATION
TECHNOLOGY
UNIVERSITI MALAYA
KUALA LUMPUR**

2021

**A COMPARATIVE ANALYSIS ON CLASS IMBALANCE
TECHNIQUES IN NETWORK TRAFFIC
CLASSIFICATION**

SHOVON DATTA RONY

**DISSERTATION SUBMITTED IN PARTIAL
FULFILMENT OF THE REQUIREMENTS FOR THE
DEGREE OF MASTER OF COMPUTER SCIENCE**

**FACULTY OF COMPUTER SCIENCE AND
INFORMATION TECHNOLOGY
UNIVERSITI MALAYA
KUALA LUMPUR**

2021

UNIVERSITI MALAYA
ORIGINAL LITERARY WORK DECLARATION

Name of Candidate: Shovon Datta Rony (I.C/Passport No: EJ0100423)

Matric No: 17050830/1

Name of Degree: Master of Computer Science (Applied Computing)

Title of Project Paper/Research Report/Dissertation/Thesis ("this Work"): A Comparative Analysis on Class Imbalance Techniques in Network Traffic Classification.

Field of Study: Computer Science

I do solemnly and sincerely declare that:

- (1) I am the sole author/writer of this Work;
- (2) This Work is original;
- (3) Any use of any work in which copyright exists was done by way of fair dealing and for permitted purposes and any excerpt or extract from, or reference to or reproduction of any copyright work has been disclosed expressly and sufficiently and the title of the Work and its authorship have been acknowledged in this Work;
- (4) I do not have any actual knowledge nor do I ought reasonably to know that the making of this work constitutes an infringement of any copyright work;
- (5) I hereby assign all and every right in the copyright to this Work to the University of Malaya ("UM"), who henceforth shall be owner of the copyright in this Work and that any reproduction or use in any form or by any means whatsoever is prohibited without the written consent of UM having been first had and obtained;
- (6) I am fully aware that if in the course of making this Work, I have infringed any copyright whether intentionally or otherwise, I may be subject to legal action or any other action as may be determined by UM.

Candidate's Signature: 

Date: 21/12/2021

Subscribed and solemnly declared before,

Witness's Signature:


Assoc. Prof. Dr. Nor Badrul Anuar Bin Juma'at
Department of Computer System and Technology
Faculty of Computer Science & Information Technology
University of Malaya
50603 Kuala Lumpur

Date: 21 December 2021

Name: Associate Prof Ts. Dr. Nor Badrul Anuar Bin Juma'at

Designation: Supervisor

A COMPARATIVE ANALYSIS ON CLASS IMBALANCE TECHNIQUES IN NETWORK TRAFFIC CLASSIFICATION

ABSTRACT

Classifying network traffic continues to evolve, with the number of network-connected applications increasing rapidly over the last few years. As a result, network monitoring related task becomes convoluted. Network traffic classification helps manage network monitoring by classifying network traffic into different classes, thus giving network administrators an in-depth overview. Traffic classification using machine learning is the most popular approach currently. Few challenges are involved in traffic classification using machine learning. One of the most challenging issues associated with machine learning is class imbalance. Class imbalance is a situation where each class has an imbalanced proportion of data samples. Despite more than two decades of continuous development, learning from imbalanced data is a focus of reckless research. Imbalance class distribution leads to the wrong prediction of a classification model. Traditional machine learning algorithm assumes that the number of samples belonging to each class is equal, producing low sensitivity and detecting minority classes as a majority. Research on class imbalance now grows higher as classification using machine learning becomes more straightforward than earlier studies. Most studies related to network traffic classification using machine learning aim to classify network traffic into various categories. Studies extend the machine learning application to several classification areas, including face recognition, text classification, anomaly detection, and medical diagnosis. Despite those studies, only a few works addressed the class imbalance problem in network traffic classification. Literature shows that people miss class imbalance in network traffic classification and only focus on overall accuracy. There are two types of imbalanced problems, binary and multiclass. Most studies focused on the binary class imbalanced

problems only. However, in real-world network traffic, the classification might have more than two classes, and most probably, data distribution among the classes are imbalanced. Multi-class imbalance problems are more complex situations that need more effort than binary class imbalance. This study focuses on examining how the various data balancing techniques affect the sensitivity of minority classes in a multiclass classification problem with a highly imbalanced dataset to provide the importance of detection sensitivity on minority classes. In order to compare and analyse the performance, this study uses an extensive public dataset from Kaggle and Random forest classification for the evaluation. The dataset has 87 features where each of the instances has the information of IP flow, including the application names. This study compares the data balancing performance using seven (7) undersampling and oversampling techniques. The techniques include random undersampling, ENN, random oversampling, SMOTE, ADASYN, SmoteENN, and SmoteTomek. Results reveal that different data balancing techniques affect the classification of minority classes differently, with some improving performance while others were degrading it. Among all the comparisons of each technique, algorithms perform better: Random Oversampling, SMOTE, SmoteENN, and SmoteTomek as for the minority class.

Keywords: Class Imbalance, Machine Learning, Network Traffic Classification.

ANALISIS PERBANDINGAN TEKNIK KETIDAKSEIMBANG KELAS DALAM KLASIFIKASI TRAFIK RANGKAIAN

ABSTRAK

Mengelaskan trafik rangkaian terus berkembang, dengan bilangan aplikasi yang disambungkan rangkaian meningkat dengan pesat sejak beberapa tahun kebelakangan ini. Akibatnya, tugas berkaitan pemantauan rangkaian menjadi berbelit. Klasifikasi trafik rangkaian membantu mengurus pemantauan rangkaian dengan mengklasifikasikan trafik rangkaian ke dalam kelas yang berbeza, sekali gus memberikan pentadbir rangkaian gambaran keseluruhan yang mendalam. Pengelasan trafik menggunakan pembelajaran mesin ialah pendekatan yang paling popular pada masa ini. Beberapa cabaran terlibat dalam klasifikasi trafik menggunakan pembelajaran mesin. Salah satu isu paling mencabar yang dikaitkan dengan pembelajaran mesin ialah ketidakseimbangan kelas. Ketidakseimbangan kelas ialah situasi di mana setiap kelas mempunyai perkadaran sampel data yang tidak seimbang. Walaupun lebih daripada dua dekad pembangunan berterusan, pembelajaran daripada data yang tidak seimbang adalah fokus penyelidikan melulu. Pengagihan kelas yang tidak seimbang membawa kepada ramalan yang salah bagi model pengelasan. Algoritma pembelajaran mesin tradisional mengandaikan bahawa bilangan sampel kepunyaan setiap kelas adalah sama, menghasilkan kepekaan rendah dan mengesan kelas minoriti sebagai majoriti. Penyelidikan tentang ketidakseimbangan kelas kini berkembang lebih tinggi kerana klasifikasi menggunakan pembelajaran mesin menjadi lebih mudah daripada kajian terdahulu. Banyak kajian berkaitan klasifikasi trafik rangkaian menggunakan pembelajaran mesin bertujuan untuk mengklasifikasikan trafik rangkaian kepada pelbagai kategori. Kajian meluaskan aplikasi pembelajaran mesin ke pelbagai bidang pengelasan, termasuk pengecaman muka, pengelasan teks, pengesanan anomali dan diagnosis perubatan. Walaupun kajian tersebut, hanya beberapa karya yang menangani masalah ketidakseimbangan kelas dalam klasifikasi trafik rangkaian. Literatur

menunjukkan bahawa orang terlepas ketidakseimbangan kelas dalam klasifikasi trafik rangkaian dan menumpukan pada ketepatan keseluruhan sahaja. Terdapat dua jenis masalah tidak seimbang, binari dan multiclass. Kebanyakan kajian tertumpu kepada masalah ketidakseimbangan kelas binari sahaja. Walau bagaimanapun, dalam trafik rangkaian dunia sebenar, klasifikasi mungkin mempunyai lebih daripada dua kelas, dan kemungkinan besar, pengedaran data antara kelas adalah tidak seimbang. Masalah ketidakseimbangan berbilang kelas adalah situasi yang lebih kompleks dan memerlukan lebih banyak usaha berbanding ketidakseimbangan kelas binari. Kajian ini memberi tumpuan kepada mengkaji bagaimana pelbagai teknik pengimbangan data mempengaruhi sensitiviti kelas minoriti dalam masalah klasifikasi berbilang kelas dengan dataset yang sangat tidak seimbang untuk memberikan kepentingan sensitiviti pengesanan pada kelas minoriti. Untuk membandingkan dan menganalisis prestasi, kajian ini menggunakan set data awam yang luas daripada klasifikasi hutan Kaggle dan Rawak untuk penilaian. Set data ini mempunyai 87 ciri di mana setiap kejadian mempunyai maklumat aliran IP, termasuk nama aplikasi. Kajian ini membandingkan prestasi pengimbangan data menggunakan tujuh (7) teknik pensampelan terkurang dan pensampelan berlebihan. Teknik tersebut termasuk pensampelan terkurang rawak, ENN, pensampelan berlebihan rawak, SMOTE, ADASYN, SmoteENN, dan SmoteTomek. Keputusan mendedahkan bahawa teknik pengimbangan data yang berbeza mempengaruhi klasifikasi kelas minoriti secara berbeza, dengan beberapa meningkatkan prestasi manakala yang lain merendharkannya. Di antara semua perbandingan setiap teknik, algoritma berprestasi lebih baik ialah Random Oversampling, SMOTE, SmoteENN, dan SmoteTomek bagi kelas minoriti berkenaan.

Kata Kunci: Ketidakseimbangan Kelas, Pembelajaran Mesin, Klasifikasi trafik Rangkaian.

ACKNOWLEDGEMENTS

I begin by thanking the Almighty God for blessing me with good health, strength, courage, and wisdom throughout my master's study.

I feel immense pleasure to extend my sincerest gratitude to my supervisors, Associate Prof. Ts. Dr. Nor Badrul Anuar Bin Juma'at, and Prof. Ts. Dr. Miss Laiha Binti Mat Kiah, for the invaluable support and encouragement you have always extended to me through this research. Thank you for all the experiences and information shared during the supervision phase. Your endless guidance has availed me the opportunity to broaden my knowledge while giving me a sense of direction for future challenges. May the Almighty grant your good health and always bless your work.

To my beloved father Ranjit Kumar Datta, mother Arati Rani Datta and siblings, I can't thank you enough for the invaluable love, support and encouragement you have tirelessly extended to me during my study. No words can express my feelings and how grateful I'm for having people like you in my life. May God always guide and protect you.

I also would like to thank my friends who were there in time of need. I learned together with them; they are fantastic as classmates and make a lot of difference during the time of my study. I am taking this opportunity to thank each and every one of them for being kind to share their knowledge and skills with me.

Last but not least, thanks to all UM FSKTM lecturers and staff for their support and care for all FSKTM students. Thanks to them for providing a comfortable, nice study environment and facilities for students like me so that we can focus on our studies.

TABLE OF CONTENT

Abstract	iii
Abstrak	v
Acknowledgements	vii
Table of Content.....	viii
List of Figures	x
List of Tables.....	xi
List of Abbreviations.....	xii
CHAPTER 1: INTRODUCTION	1
1.1 Background	1
1.2 Problem Statement	8
1.3 Research Motivation.....	9
1.4 Research Objectives	10
1.5 Research Scope.....	11
1.6 Research Significance	11
1.7 Thesis Structure	12
CHAPTER 2: LITERATURE REVIEW.....	13
2.1 Introduction	13
2.2 Network traffic classification approaches	13
2.3 Random forest classifier	19
2.4 Class imbalance Problem	20
2.5 Type of approaches to handle class imbalance	21
2.5.1 Data Level	22
2.5.2 Algorithm Level	26
2.5.3 Cost-sensitive Level	28
2.6 Conclusion.....	32
CHAPTER 3: METHODOLOGY.....	33
3.1 Introduction	33
3.2 Research Methodology	33
3.3 Experimental Methodology	37
3.3.1 Dataset.....	37
3.3.2 Data Processing.....	39
3.3.2.1 Creating training and test datasets	40
3.3.3 Machine Learning	41

3.3.4	Evaluation	42
3.4	Environmental Setup	45
3.5	Conclusion.....	46
CHAPTER 4: RESULTS AND DISCUSSIONS		47
4.1	Introduction	47
4.2	Distribution of classes in the dataset	47
4.3	Classification without Imbalance Technique	48
4.4	Classification with Imbalance Techniques.....	50
4.4.1	Random Undersampling.....	50
4.4.2	Edited Nearest Neighbor (ENN)	52
4.4.3	Random Oversampling.....	53
4.4.4	Synthetic Minority Oversampling Technique (SMOTE).....	55
4.4.5	Adaptive Synthetic Sampling (ADASYN)	56
4.4.6	Synthetic Minority Oversampling Technique with Edited Nearest Neighbor (SmoteENN)	58
4.4.7	Synthetic Minority Oversampling Technique with Tomek Links (SmoteTomek)	59
4.4	Conclusion.....	63
CHAPTER 5: CONCLUSION.....		65
5.1	Introduction	65
5.2	Achievement of research objectives.....	65
5.3	Contribution of the research	66
5.4	Limitations of the research	66
5.5	Future work	67
5.6	Conclusion.....	68
REFERENCES.....		69

LIST OF FIGURES

Figure 2.1: Types of approach for class imbalance.....	22
Figure 3. 1: Phases of the general research methodology	33
Figure 3. 2: SMART Method	35
Figure 3. 3: Flow of experiment.....	37
Figure 4. 1: Distribution of classes	48
Figure 4. 2: Sensitivity of minority class in dataset	49
Figure 4. 3: Comparison of classification sensitivity pre and post Random Undersampling	51
Figure 4. 4: Comparison of classification sensitivity pre and post ENN	53
Figure 4. 5: Comparison of classification sensitivity pre and post Random Oversampling	54
Figure 4. 6: Comparison of classification sensitivity pre and post SMOTE.....	56
Figure 4. 7: Comparison of classification sensitivity pre and post ADASYN	57
Figure 4. 8: Comparison of classification sensitivity pre and post SmoteENN.....	59
Figure 4. 9: Comparison of classification sensitivity pre and post SmoteTomek.....	61
Figure 4. 10: Overall accuracy for pre and post application of each algorithm.....	62
Figure 4. 11: F1-score for pre and post application of each algorithm	63

LIST OF TABLES

Table 2. 1: Previous key studies on class imbalance	31
Table 2. 2: Previous key studies on class imbalance (continued)	32
Table 3. 1: List of applications in the dataset	38
Table 3. 2: Categories of application in the dataset	39
Table 3. 3: Categories of application in the dataset (continued)	40
Table 3. 4: Confusion matrix for an example that is predicted positive or negative	43
Table 4. 1: Performance of Random Forest classifier prior to any balancing technique	49
Table 4. 2: Classification performance after deploying random undersampling technique.	51
Table 4. 3: Classification performance post deploying ENN	52
Table 4. 4: Classification performance post deploying Random Oversampling technique.	54
Table 4. 5: Classification performance post deploying SMOTE	55
Table 4. 6: Classification performance post deploying ADASYN	57
Table 4. 7: Classification performance post deploying SmoteENN	58
Table 4. 8: Classification performance post deploying SmoteTomek	60
Table 4. 9: Sensitivity of each algorithm applied	61

LIST OF ABBREVIATIONS

ADASYN	:	Adaptive Synthetic
AI	:	Artificial Intelligence
AUC	:	Area Under Curve
BSMOTE	:	Borderline Synthetic Minority Oversampling Technique
CART	:	Classification and Regression Trees
CSL	:	Cost-Sensitive Learning
CSV	:	Comma Separated Value
DL	:	Deep Learning
DNS	:	Domain Name System
DPI	:	Deep Packet Inspection
DT	:	Decision Tree
ENN	:	Edited Nearest Neighbor
FCM	:	Flow Based Matrix
FTP	:	File Transfer Protocol
HTTP	:	Hyper Text Transport Protocol
HTTPS	:	Hyper Text Transport Protocol Secure
IANA	:	Internet Assigned Numbers Authority
ICMP	:	Internet Control Message Protocol
IMAP	:	Internet Message Access Protocol
IDGC	:	Imbalanced Data Gravitation Based Classification

KNN	:	K-nearest Neighbors
ML	:	Machine Learning
NB	:	Naïve Bayes
NTC	:	Network Traffic Classification
P2P	:	Peer to Peer
RF	:	Random Forest
ROS	:	Random Oversampling
RTP	:	Real-time Transport Protocol
RUS	:	Random Undersampling
SMOTE	:	Synthetic Minority Oversampling Technique
SSL	:	Secure Sockets Layer
SVM	:	Support Vector Machines
VoIP	:	Voice over Internet Protocol
WCM	:	Weighted Cost Metrics
WSU	:	Weighted Symmetric Uncertainty
WWW	:	World Wide Web

CHAPTER 1: INTRODUCTION

This chapter presents an introduction to the research work. It highlights numerous aspects related to the research work. Moreover, this section provides a general background about the research area followed by the problem statement, research motivation, objectives, and scope of research. Finally, the end of this section explains the significance of the study to the networking field in general.

1.1 Background

Classifying the network traffic is very important for efficient network management and monitoring systems. As the transmission speed is growing higher and faster nowadays, it becomes challenging to categorize each network flow, which increases the low performance of network operation. A defective traffic classification can hamper the network services like the quality of service, network scalability, bandwidth distribution, intrusion detection, and access control. In computer science, network traffic classification (NTC) is now one of the prime topics. Recently, network researchers have been focusing their interest on enhancing traffic classification accuracy (Khalife, Hajjar, & Diaz-Verdejo, 2014).

Various network traffic classification techniques have been proposed and deployed earlier to classify traffic. Network traffic can categorize into three types (Callado, Kamienski, Szabó, & al., 2019; Nguyen & Armitage, 2008): port-based classification, deep packet inspection (DPI), and machine learning. The earlier classification methods were port-based classification and Deep Packet Inspection (DPI). DPI inspects the whole packet header and payload from layer 2 to layer 7 to classify the packet. It allows DPI to be aware of the packet contents. However, these two methods are now less effective at present. Port-based classification has become obsolete because nowadays, the application hides ports or use dynamic ports to mask their communication from attackers. The

Internet Assigned Numbers Authority (IANA) reserved most well-known ports; hence, the port-based method cannot identify most traffic (Sen, Spatscheck, & Wang, 2004).

On the other hand, DPI, also known as a payload-based classification, violates user data privacy. 3rd parties can easily see the traffic content, which is very harmful to many kinds of industry. DPI is inappropriate for deployment in cases involving vast amounts of data. Moreover, DPI cannot inspect encrypted data (Callado, Kamienski, Szabó, & al., 2019). Because of these limitations, as mentioned above, many researchers found an efficient way to classify the traffic based on the flow of statistical features. Machine Learning takes advantage of the statistical features available in packet headers, so it becomes a hot research topic (Liu Q, Liu Z, & M., 2010). Consequently, machine learning removes the need to inspect the packet payload, which raises privacy concerns among the community.

Classification using machine learning removes the hassle and improves further from the previous methods. Machine learning is a sequence of algorithms that describe data, acquire information from it, and use what it has learned to produce intelligent decisions. The learning process begins with data or observations, such as examples, instructions, or knowledge, to seek patterns in data and make better resolutions in the future depending on the provided examples. The elementary objective is for computers to study independently, without involving human interaction, and to change their behaviour in accordance with the knowledge they gained (Singh, Thakur, & Sharma, 2016). Therefore, it is an artificial intelligence (AI) application that allows computers to gather the knowledge related to data and improve its performance from given example data without the need for manual programming.

Few challenges involve in machine learning; one of the most challenging is class imbalance. With a slight imbalance problem, the entire issue can become a serious

challenge for evaluation (Elazmeh W., Japkowicz N., & S., 2006). Class imbalance is a situation where the instance of one or more classes is over or underrepresented in a dataset. For example, a network dataset contains information of 100 network flows, whereas FTP has only one packet. On the other hand, HTTP has got 99 packets. This situation is called a class imbalance. If someone wanted to train the imbalanced dataset to classify network protocol automatically, he would get the accuracy of 99% just by correctly classifying HTTP. In that case, if the classification algorithm misclassifies 1% of FTP, it can make the network vulnerable. And imagine if the dataset were large or big data, classification algorithm would produce misleading accuracy. Another way of describing the imbalance class in a dataset is to recap the class distribution according to the percentages of each available class in the training dataset. For instance, an imbalanced multi-class classification problem can include 82 per cent of examples in the first class, 16 per cent of examples in the second class, and 2 per cent of examples in a third class. The imbalanced distribution of class varies across different problems. May a classification problem be slightly skewed, like a bit of imbalance only. On the other hand, the classification issue may have an extreme imbalance. There could be hundreds or thousands of examples just in one class for a given training dataset, whereas tens of examples in another class.

- a) Severe Imbalance: it is a kind of imbalanced classification issue in which the distribution of samples in the training dataset is unequal or has impartiality by a large difference. (e.g. 1:100).
- b) Slight Imbalance: it is a kind of imbalanced classification issue in which the distribution of examples in the training dataset is unequal or has impartiality by a small amount. (e.g. 8:10 or even more).

Usually, a slight imbalance is less concern, and the concernment may regularly treat like a typical predictive modelling problem for classification. Modelling a severe class imbalance may have difficulties and may demand the work of specialized techniques.

The cause of class imbalance in a classification predicting model can occur for some reasons. It is most likely that it may cause during the collection of the data or the way of collecting the data. One of the causes could be labelling the wrong class and repeating the same in the other classes of the dataset. Nonetheless, the imbalance problem might be a substance of the problem territory. One class's natural existence or presence, for example, may dominate the other classes. The issue may arise if the process of generating observations in one class is more expensive in terms of computation, cost, time, or other resources. So, collecting more samples from a domain to construct a better class distribution is sometimes difficult or hardly possible. Instead, a model is needed to know the distinctions between the classes. Sometimes, in some cases, the imbalance is occurred by measurement error or a sampling bias. This situation occurs in almost all network environments as some services or applications are often consumed more than others, thus creating a non-uniform class distribution. Non-uniform class distribution negatively affects learning algorithms. The distribution can range from a little imbalance to an extreme bias where only one example is in the minority class against hundreds, thousands, sometimes millions in the majority class. The imbalanced classification holds a provocation for predictive classification modelling since most machine learning algorithms expect the same amount of examples for individual classes available in the dataset. As a result, models with poor prediction accuracy, particularly for the minority class, emerge. As such, the community requires a solution that can tackle class imbalance to improve the performance of the learning algorithms. However, the imbalance issue can solve by correcting the measurement error and improving the sampling method. The training dataset is an unclear delegation of the problem that addresses the imbalance.

Class imbalance in a classification model can be slightly imbalanced or highly imbalanced. Sometimes slightly imbalanced data is not a concern but still depends on the domain. Highly imbalanced data is always in concern and need to treat with special techniques.

“Any dataset with an unequal class distribution is technically imbalanced. However, a dataset is said to be imbalanced when there is a significant, or in some cases extreme, disproportion among the number of examples of each class of the problem.” – Learning

From Imbalanced Data Sets, (Fernández, et al., 2018, p. 19).

Predictive modelling for a classification problem can contain only two class labels, such as either 0 or 1. The modest classification problem can indicate a binary or a two-class classification. On the other hand, the problem may have more than two classes, like three classes, ten classes, or even hundreds. These sorts of issues are alluded to as multi-class classification problems.

- a) Binary Classification: One kind of classification model that needs to predict which example belongs to what class from a dataset with only two classes.
- b) Multiclass Classification: One kind of classification model that needs to predict which example belongs to what class from a dataset with three or more classes.

A classification model using machine learning requires gathering a training dataset to understand and learn about the data. A training dataset is a collection of several samples from a domain that includes both input data such as measurement and output data such as a class label. Based on the seriousness of the problem and the types of models, a study that applies a training dataset might consist of tens, hundreds, thousands, or even millions of samples from the domain.

The class with a vast amount of examples is called the majority or major class. In contrast, the minority or minor class distinguishes a small number (sometimes only one) of examples.

- a) Majority Class: the class(es) holds the most examples in a dataset.
- b) Minority Class: the class(es) that holds only fewer examples in a dataset

The minority class is generally the most interesting when dealing with an imbalanced dataset. In a predictive classification, a model's ability to properly forecast the minority class label or probability is more essential than the majority class(es).

The most popular way of handling class imbalance is sampling (More, 2016). Various sampling methods are available to deal with an imbalanced dataset, and these methods can categorize into three types: resampling, cost-sensitive learning, and algorithm level. The nature of these methods is to modify the class proportion and size of the actual dataset, and sometimes these methods can combine with other methods to form a hybrid method. It is quite common to balance the major and minor classes in the datasets by adding additional weights in the minor class or removing extra weights from the major classes. Therefore, oversampling can cause overfitting because it copies the same existing instances. On the other hand, while eliminating the additional weights, under-sampling can lead to the loss of important information. Still, most researchers prefer the sampling method over cost-sensitive learning.

The distribution of data samples is one of the primary factors that determine the goodness of the classification performance of any machine learning algorithm. Even distribution of data samples among the classes produces good results and vice-versa. This attribute is that these algorithms develop to maximize overall accuracy and minimize errors. There are two types of class for classification, and they are called binary class and

multi-class. The minority class(es) is more difficult to anticipate since this group has fewer instances compared to the majority class(es). That indicates learning the properties of instances from the minority classes and distinguishing them from the majority classes.

The excessive amount from majority classes examples can inundate the minority classes examples. Nearly all predictive classification models using machine learning methods are built and presented in situations considering a fair distribution of each class. That means a naive model classification may target exclusively on learning the nature and attributes of the large observations only, avoiding the instances from the minority classes, which are of more importance and whose predictions are more worthy. In binary class classification, there is only two possible class, such as either 1 or 0. A classification with more than two classes is called multiclass classification. In the real world, multi-class classification for imbalanced data is quite common. NTC is being categorized under multi-class classification because classifiers aim to classify based on different kinds of applications or protocols. Few researchers captured traffic data for classification to identify the type of application that contained in IP flow such as telnet, FTP, P2P, WWW, DNS, and so on (Zhen & Qiong, 2012; Liu, Wang, Tao, & Cai, 2015; Jie Yang, Yixuan Wang, Chao Dong, & Cheng, 2012). This study collects a dataset to identify specific types of applications like Instagram, Google, YouTube, Facebook, Skype, etc., from the flow of statistics. One of the issues is that traffic classifiers based on machine learning may achieve low accuracy for the minority classes.

The remaining parts of chapter 1 describe the problem statement, research motivation, research objectives, scope, the significance of the research, and the thesis structure described. The study's literature review and the subsidiary information discusses in chapter 2. Chapter 3 demonstrates how this research carry out based on the research

process, dataset, and experiment. Chapter 4 presents results and discussion in detail. Finally, chapter 5 of this document concludes the whole research.

1.2 Problem Statement

Class imbalance is a typical problem in network traffic classification using machine learning. Most of the studies conducted on class imbalance used two-class datasets. The multi-class imbalance has always been more difficult to handle when it is about the multi-class classification and requires more knowledge and arithmetical techniques. In the real world scenario, the network traffic is imbalanced and unequally distributed to each type of traffic. Suppose an organization captures the network traffic for analysis; they cannot expect all the network packets to be the same size or equally coming from different sources. Conventional machine learning algorithms assume that the data set has balanced class distributions, thus producing a high classification accuracy and good performance. Class imbalance gives a false assumption that the model performs well with high accuracy (Ali, Shamsuddin, & Ralescu, 2015).

Machine learning algorithms classify instances wrongly and detect minority as a majority which directs the classification result toward the low sensitivity. Sometimes minority class is more important and interesting too. For example, the number of intrusions or malicious traffic is significantly less when compared to overall network traffic. That can leave the whole network vulnerable if such traffic comes from a malignant source, thus leading to wrong business decisions by network administrators.

As such, this study attempts to research various methods to deal with the imbalance distribution of class in network traffic classification with specific application classes.

1.3 Research Motivation

The rising trend towards traffic encryption, together with encapsulating the network protocol, demands further challenge and motivates to conduct the study comprehensively. The class imbalance in network traffic classification is the most recent study conducted by a few researchers to address the imbalance because of the imbalanced dataset. It is becoming a pointy trend to fix the class imbalance issue in network traffic classification. Only few people recently started exploring this topic since 2017 (Gómez, Hernández-Callejo, Martínez, & J.Sánchez-Esguevillas, 2019; Peng, Yang, Chen, & Abraham, 2009; Gómez, Martínez, Sánchez-Esguevillas, & Callejo, 2017; Xiao, et al., 2019).

Literature shows that people miss class imbalance in network traffic classification and focus on accuracy only (Kwon, Jung, & Park, 2020; Kumar, Venkanna, & Tiwari, 2020; Nazarenko, Varkentin, & Polyakova, 2019; Kwon, Lee, Yu, & Park, 2020).

Only accuracy alone cannot depict the true result of classification. The accuracy prediction can be higher even 99%, but the question remains how much example has correctly classified the example of class where it truly belongs? According to a recent study, it stated that:

“in the framework of imbalanced data-sets, accuracy is no longer a proper measure, since it does not distinguish between the numbers of correctly classified examples of different classes. Hence, it may lead to erroneous conclusions” (Maheshwari, Jain, & Jadon, 2017).

Owing to the points mentioned above, it can conclude that still there is a need of exploring the class imbalance problem. Not just network traffic classification, there are many other fields currently in this domain where this study can contribute. In the class distribution, although many classification issues have a significant imbalance in the class

distribution, looking at frequent problem domains that are naturally imbalanced would help to concretize the principles and challenges of class imbalance fact. The superiority of class imbalance has several other domains to apply. Some important fields are network intrusion detection, medical diagnosis, direct marketing, fraudulent transactions, fraud detection, spam detection, image recognition, claim prediction, default prediction, churn prediction, conversion detection, oil spill recognition, outlier detection. The examples mentioned above clears up the presence of predictive imbalanced classification modelling. Individual issues may be wrapped and analyzed as predictive imbalanced classification modelling in each of these problem areas, which comprises the study of an entire field. That features the interdisciplinary nature of class imbalanced classification, which is why a machine learning specialist must be apprehensive of the issue and be proficient in discussing it.

1.4 Research Objectives

Based on the research problem outlined in section 1.2, this research examines various methods to deal with class imbalance distribution in network traffic classification. To fulfil the aim of this research, it comprises the following objectives:

- i. To study the impact of class imbalance in network traffic classification.
- ii. To investigate the mode of operation of the various techniques commonly used for data balancing in network traffic classification.
- iii. To evaluate the advantages and disadvantages of the data balancing model in terms of performance based on an imbalanced dataset.

1.5 Research Scope

Class imbalance study has a broad scope in machine learning such as intrusion detection, fraud detection, text classification, image recognition, disease diagnosis, weather forecast, etc. Although the study involves the classification of network traffic, this study is primarily concerned with overcoming the class imbalance problem by improving the sensitivity of the minority class. The desired classifier classes consist of specific applications like Instagram, Google, YouTube, Facebook, Skype, WhatsApp, Microsoft. The study implements a statistical approach with machine learning. For classification purposes, this study uses a Random Forest classifier algorithm. For dealing with the imbalanced dataset, this study deployed seven data level approaches Random Undersampling, ENN, Random Oversampling, SMOTE, Adasyn, SmoteENN, and SmoteTomek, where performance evaluated by accuracy, recall, precision, and f1score.

1.6 Research Significance

Correctly classifying network traffic flow is very important nowadays to improve the network services such as quality of service, network scalability, bandwidth distribution, intrusion detection, access control, etc. Without an efficient classification, these network services can be poorly affected. Though machine learning is an efficient way to classify with pretty high prediction, still it faces a problem when the data is highly skewed. Hence the community of network management and researcher can handle class imbalance to improve the performance of the learning algorithms. This study facilitates network management and administrator for effective decision making. Besides that, security incident handlers can benefit from this study to minimize potential risks.

1.7 Thesis Structure

The organization of the remaining part of this thesis in the following way:

Chapter 2: This chapter provides an overview of the current scene of the class imbalance problem of network traffic classification. It discusses the concept of previous works and findings on related topics to identify state of the art in class imbalance. This study includes an overview of the three main approaches to class imbalance: data level, algorithm level, and cost-sensitive level. Moreover, from the literature review, this study summarizes the most important keys of the recent research in this domain, such as the focus of research, number of dataset samples, approach, algorithm, evaluation comparison. This chapter aims to synthesize the relevant findings that have been published and reviewed in journals and proceedings.

Chapter 3: This chapter describes how the overall research was conducted by explaining the different phases and tasks performed. It reflects the sequential flow of the various activities. It begins with giving details of the research phases followed by the dataset and why the used dataset is considered the best choice for this research. It then describes the data pre-processing, applying machine learning algorithms and evaluation metrics.

Chapter 4: This chapter illustrates the final result of the experiment, along with some discussion about the results. In addition, it includes comparisons between the results of the various algorithm used in this research.

Chapter 5: This chapter is the final chapter of this research. This chapter provides a conclusive statement of what was done and achieved, limitations, and future works.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

Network traffic classification gains gradual attention from the network research community as the practice of the internet is increasing remarkably. Generally, it is a significant area in network monitoring. Correctly classifying network traffic is crucial to maintaining the quality of service of an organization or a service provider. Different methods can classify network traffic differently to get the desired output. This section summarizes previously used approaches for network traffic classification and issues related to class imbalance.

2.2 Network traffic classification approaches

Network traffic can categorize into various classes, and those classes can be the type of network (e.g., p2p), type of application (e.g., youtube, instagram), protocols (e.g., http, ftp), or the size of network packets. Researchers experimented with different types of classification in their research and used appropriate datasets for it. (Rizzi, Colabrese, & Baiocchi, 2013) Utilized HTTP, FTP, POP3, SSH, eMule, BitTorrent, eDonkey, SSL, IMAP, SMTP as classification classes using a private dataset in their study. (Camacho, Padilla, Padilla, & Díaz-Verdejo, 2013) Introduced a pairing method by inspecting the payload for traffic classification. They have used DNS, ICMP, HTTP, FTP, SSL, RTP, SSH, NTP, and 30 other applications as classification classes from the dataset. Some researchers consider application names as classification classes. The authors (Wang, Yahyavi, Kemme, & He, 2015) have collected datasets from wireless traffic mainly used by smartphones. Most people used the individual apps for each application instead of using a web browser. Smartphone's traffic can easily identify using the traffic patterns and application services because those apps generate more identical traffic patterns. Their classification classes are different application names; Chrome, Boom Beach, Youtube,

Songza, Facebook Messenger, Tcent QQ, Snapchat, Facebook, Twitter, Tinder, Mint, CDC News, Medscape. After the application name, they considered grouping application names into a few categories: browsing, gaming, multimedia, online chatting, social network, dating, financial, and medical. A paper (An, Lee, Ham, & Kim, 2015) exploring the limitations of network traffic classification caused by unusual TCP characteristics and proposed a new classification approach based on the application utilizing statistical signature to mitigate unusual TCP behaviours. This proposed approach originates a new unique signature for each application using the payload size, direction, packet order in the flow and benefits them by classifying applications from the traffic. They considered Skype, Naverlive, GomTV, Xshell, TeamViewer, Nateon, Dropbox, PuTTY, Outlook, uTorrent as application classes in their study.

In the early stage of the internet, it was more likely easy to identify applications from network traffic using the port number and packet inspection. There are three main techniques available in traffic classification: port-based, payload-based, and machine learning-based. The current state of the art for network traffic classification relies on statistical features by using machine learning. Hence, much research is already producing accurate traffic classifiers (Finsterbusch, Richter, Rocha, Muller, & Hanssger, 2014; Moore, Zuev, & Crogan, 2005; Namdev, Agrawal, & Silkari, 2015). A significant explicit review on traffic classification advances machine learning in its early years of 2008 was conducted by Nguyen and Armitage (Nguyen & Armitage, 2008). Although most of the mentioned study applies only offline, the authors also aimed to explore online deployment, accomplishing some ticklish functional requirements for the machine learning models. However, the models mentioned above are less satisfying always in a real-world application. There are many learning models to train a classifier, and several learning models handle various issues using different assumptions. Mainly these types of classifier trains to recognize features from the network traffic. For instance, network

packet size, packet length, arrival time, etc., are used to classify network traffic using clustering to similar attributes. Although various classification problems need to suppress yet to gain a proficient network traffic classifier mentioned by Dainotti et al. (Dainotti, Pescapé, & Claffy, 2012), machine learning shows the most promising results for online network traffic classification. Traffic classification based on a machine learning model is a composition of several steps, learning algorithms being trained by utilizing information obtained from the traffic capture recorded on a dataset. Building a dataset involves many interconnecting processes that influence the learning algorithm's performance.

The capability to identify network traffic by protecting users' privacy and handling encrypted network traffic are the main reasons behind why machine learning algorithms are one of the best candidates currently used for network traffic classification. In the network traffic classification system, deep learning (DL) approaches are implemented widely and considered for deploying in real-world applications (Hinton, Osindero, & Teh, 2006). Recently, a study observed that convincing assessment and precision of network traffic makes the deep learning approach a superior traffic classifier based on the meaning of conventional methodology and deep learning (Dong & Wang, 2016). To advance a few ideas concerning machine learning algorithms for traffic data, the authors in a study explored some techniques in network traffic classification (Patel, et al., 2019). Various algorithms, for example, Support Vector Machine (SVM), κ –Nearest Neighbor (KNN) and Naïve Bayes (NB) were examined in order to enlighten the techniques of machine learning algorithms by collecting features using wire shark software. Then the data was converted to an excel format, namely CSV. They trained the dataset and tested using python libraries, which help predict and compare the analysis like how Nguyen and Armitage used in their research (Nguyen & Armitage, 2008). Then the authors discovered that KNN provides accurate outcomes, among others. KNN is one of the simplest and basic ML algorithms based on the distance between data points in the vector space. It uses

lazy learning for classification and can commonly refer to as a lazy classifier. However, the major drawback of the KNN classifier is that it performs less with unbalanced data (Li, Zhang, Peng, & Yang, 2018). Recently a study was conducted to recognize the application of browser and user from real-time network traffic of educational institutions using κ –means clustering algorithm. To handle the problem of traffic assessment specially for the mode of analysis that depend on the types of traffic available in the network in most cases (Priya, Nandi, & Goswami, 2018). The authors choose Tshark software to capture data from 5 browsers, namely Chrome, Internet Explorer, Opera, Firefox, and UC Browser, and four applications, namely Facebook, YouTube, Gmail, and Flipcart, to train the classifier in their experiment.

Few researchers used the Naïve Bayes classifier to classify network traffic. NB relies on the assumption that events distribute independently. The NB classifier determines the probability that the input instance belongs to that class for each probable class to classify an input instance into a given class of the output variable. Then the instance classifies into the class that gets the highest probability (Harzevili & Alizadeh, 2018). NB classifier operates on the assumption that the probability distribution of every variable with a dataset is independent of the probability distributions of all other variables. The NB classifier implies that removing or adding a particular attribute has little effect on the other characteristics in the dataset provided the attribute class information. For a training dataset containing n -attributes, a naïve Bayes model builds on 2^n assumptions of attribute independence. However, this assumption is less applicable in a large number of real-world scenarios. As a result, the classifier's performance appreciably degrades due to biases in the estimated probabilities. According to a study (Mukherjee & Sharma, 2012), the inaccuracy of a naïve Bayes classifier brings about using three factors: bias, noise, and variance. The factors mentioned above are favouring due to many independent assumptions made on a high-dimensional dataset, noise due to the presence of irrelevant

and redundant variables, and variance. However, the NB classifier has always given significantly good classification results despite the factors.

The two most dominating learning approaches in machine learning, a) supervised learning and b) unsupervised learning. The only difference between these two is that supervised learning needs a data labelling process by acquiring prior knowledge regarding the problem to find the application from every network flow. Whereas unsupervised learning algorithms hardly require any labelling for application to every flow, they are capable of clustering classes by themselves. SVM is one of the supervised ML classifiers and can perform regression and classification. The SVM algorithm offers a high generalisation ability for unknown data. It is among the robust machine learning classifiers that can deal with linear and non-linear cases (Chen, Hsu, & Shen, 2005). The learning process output is a classification model constructed by examining features or attributes present within the training data in supervised learning. Then this classification model classifies unused or unlabeled data. Although it may be true that there is a disadvantage over supervised learning, it is less accurate when dealing with unknown data in the training phase. Most traffic classification techniques require already classified data to classify the traffic classes. The supervised ML approaches, in particular, need training data in order to construct the predictive model. Flows are represented in the training dataset by their feature values as input and the matching traffic category as output. Throughout the training phase, a function constructs to predict the traffic class for the provided feature values utilizing the flows in the training dataset. As a result, one of the most important factors affecting the supervised learner's performance is the quality of the training data because a large number of inconsistent entries in the input/output pairs has a significant impact on learning the data. In regards to the preciseness of training and test data, the volume and age of this data are essential as they change the content and properties of network traffic (Soysal & Schmidt, 2010). Unsupervised learning addresses

the burden by removing the need to have data with predefined or labelled data. Because it always utilizes the clustering concept where it never requires a training phase. Those data have similar features, unsupervised learning groups them together for clustering. The authors (Kurt, et al., 2012) explored unsupervised learning to classify 9681 traffic flows into three categories: Video, VoIP, and P2P. The authors used a Bayesian clustering approach to create a group of traffic flows into the mentioned groups and performed with high accuracy. Although there was no explicit interpretation of the outcomes, the study managed to classify P2P traffic with near-perfect accuracy. The utility of unsupervised learning is vindicated by (Goss & Nitschke, 2013) using automated signature generation. Their study implemented a Hierarchical Self-Organizing Map (HSOM) to cluster by itself into seven protocols: POP3, SMTP, IMAP, HTTP, HTTPS, Soul-Seek, and BitTorrent. Except for HTTP, they acquired more than 99% accuracy on other protocols. They have claimed that the presence of HTTP pipelining concerned the classifier to misclassify the protocol.

Many researchers have adopted one of these two learning approaches for network traffic classification. Some researchers used both supervised and unsupervised learning together, known as semi-supervised learning or hybrid learning. In other words, it uses a mixture of both labelled and unlabelled samples to build a classifier. Semi-supervised learning also shows promising results. Semi-supervised learning requires only a small amount of pre-labelled traffic flow in network traffic classification. A semi-supervised network traffic classification method was recently proposed based on $\chi - means$ clustering algorithm and new label propagation for classifying large volume of data input (Exposito, Gineste, Baudoin, & Aguilar, 2019). $\chi - means$ is a kind of k-means algorithm that deals with the allocations of clusters by constantly trying and keeping the resultant appeasement divides until it reaches the standard. This approach involves pre-processing of data, conversion from the nominal value to the numerical value, significant

feature selection, clustering, label propagating, and finally, the outcome and analysis of the classification. Generally, all three learning (supervised, unsupervised, and semi-supervised) methods expect complete and readily available datasets. However, in the real network environment, traffic departs and arrives continuously. Hence, the dataset becomes chaotic and large by size, which requires proper training. This study uses a supervised learning method based on a decision tree (DT), known as Random Forest. A decision tree is a typical machine learning algorithm that generates decisions using a "tree structure". The decision tree is simple to comprehend because of its straightforward hierarchy and processing procedures. A decision tree usually has one root node, a few internal nodes, and a few leaf nodes. Each node's samples make partition into sub-nodes based on the results of the attribute test, with the root node containing the whole sample set. A decision test sequence corresponds to the path from the root node to the last leaf node. Decision tree training is a recursive procedure that follows a basic and obvious "divide and conquer" technique. (Chang, et al., 2019).

2.3 Random Forest classifier

A random forest classifier is an ensemble of many decision trees trained using the bagging method. It provides good results for binary class classification problems and multi-class classification problems (Labayen, Magaña, Morató, & Izal, 2020; Tripathi, Goswami, Trivedi, & Sharma, 2021; Mohana, Reddy, Anisha, & Murthy, 2021). In many cases, single classifiers have hardly yielded reliable predictions. Many classification applications are based on a collection of base-classifiers to form what is known as an ensemble classifier. Random Forest is one of the most efficient and reliable ensemble classifiers based on many decision trees. The final class of any input instance is determined based on the majority vote, where every tree casts one vote (Breiman, 2001). Multiple unpruned trees generate to train a random forest classifier, with each particular tree being trained independently on a bootstrap sample drawn from the input training

dataset. RF uses bagging, where a random sample draws from the original dataset with a replacement for each unpruned tree. All unpruned trees only stop growing when they achieve the maximum depth. Sometimes pruning is used to prevent the growth of a DT. That ables to achieve by defining a stopping condition based on the resources or performance or explicitly defining a maximum tree depth. Pruning helps to control tree complexity and reduces the chances of over-fitting. In addition, RF naturally performs some feature selection by examining each attribute's relative importance, which is a vital step for dimensionality reduction (Prashanth, Prashanth, Jayashree, & Srinivasan, 2008). To obtain the relative importance of a particular attribute, RF removes the input variable while keeping others and computes either the average increase in the classification error or the average decrease in the classification accuracy. It then assigns a rank value to each feature, indicating its importance in classifying input instances.

2.4 Class imbalance Problem

Class imbalance problems can be categorised into binary class imbalance problems and multi-class imbalance problems. The binary class imbalance problem has only two classes where each instance considers either majority class or minority class. Most of the studies use binary class imbalanced problems in the literature. In real life, classification has more than two classes, and most probably, skewed data distribution among the categories. Multi-class imbalance problems are in more difficult situations and need more effort than binary class imbalance problems because the relationship among the classes is less straightforward (IV & Krawczyk, 2021). The recent tendency in imbalance learning shows there is a need to explore the characteristics of each class and instance belonging to the dataset (Sáez, Krawczyk, & Woźniak, 2016). In other words, the imbalance study should analyze the ratio of the imbalance class and the characteristics of every class. Conventional machine learning algorithms presume that the instances of different classes are equal. Hence, most machine learning classifier algorithm builds to

maximize the accuracy as much as possible and reduce error. Indeed, most of the time, the distribution of instances becomes skewed because some of the instances appear very often than others, and this kind of situation is very common. For example, suppose there is a classification model where the dataset has four labels that to be classified. The classes are web browsing with 70% volume, streaming with 20% volume, social media with 5% volume, and chat with 5% volume. This study can see that the dataset is highly imbalanced from this example. This kind of situation is very frustrated when this study found that data has imbalanced classes and, as a result, achieved high accuracy becomes useless. However, there is a lack of studies on the classification of highly imbalanced datasets. Erman et al. focused on the class imbalance issue on NTC for the very first time in NTC (Erman, Mahanti, & Arlitt, 2007). The authors proved how this imbalance distribution of network data influences NTC's precision. They also suggested using byte accuracy to evaluate the performance of the classification model.

2.5 Type of approaches to handle class imbalance

The study finds various methods to propose a different solution for tackling the multi-class imbalance problem. There are currently three primary methods in this process: data level, algorithm level, and cost-sensitive level-based approaches. Data level focus on modifying the training set to make it appropriate for a standard algorithm. The algorithm level focuses on modifying the existing learning algorithm to mitigate their favour towards the majority class. The cost-sensitive level balances the cost of errors (misclassification) in the dataset during pre-processing or post-processing. Generally, there are three types of approaches for class imbalance problem, and these are:

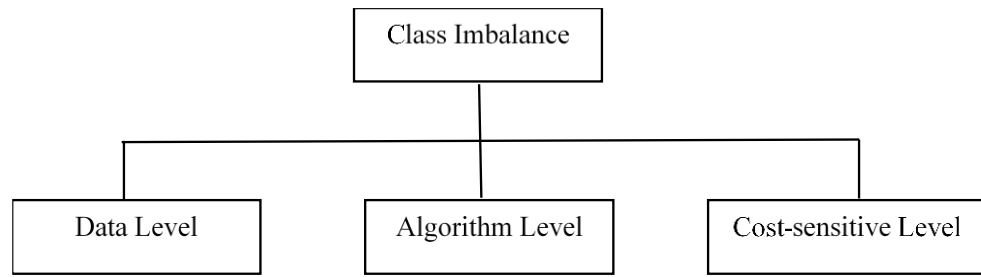


Figure 2.1: Types of approaches for class imbalance

2.5.1 Data Level

Data level methods resolve class imbalance by modifying class distributions before or during the training. This method can refer to resampling as it resamples the data differently. The resampling technique aims to build an oversampling and undersampling technique to tackle imbalanced class distribution in the original dataset. Since the beginning, resampling techniques seem to be the most promising solution to the class imbalance problem. Ajinkya More reviewed several resampling techniques to handle imbalanced datasets and study their effect on classification performance (More, 2016).

Undersampling: undersampling is where some samples of the majority class are removed from the dataset so that the number of samples is equal for all classes. This technique aims at balancing the majority and minority class distributions. Among the undersampling techniques, some popular approaches are Random undersampling (Batista, Prati, & Monard, 2004), Edited Nearest Neighbors (Wilson, 1972), Near Miss (Zhang & Mani, 2003), and Tomek Links (Tomek, 1976). Edited Nearest Neighbors can either belong to the majority class or minority class (ei), having different labels with all three nearest neighbours. If (ei) belongs to the minority class, then three nearest neighbours of (ei) are removed, and if (ei) belongs to the majority class, then (ei) is removed. Edited Near Miss approach works based on near-miss version. It uses the average distance of 3 farthest and nearest minority classes by selecting those majority

classes. Random undersampling (RUS) is an approach where random instances of the majority class eliminate until the number of instances for each class is equal to the number of instances in the class with the least number of instances (Batista, Prati, & Monard, 2004). This approach has the advantage of improving system resource requirements like processing or training time and memory as it lowers the number of training examples in cases where the training dataset is large. However, due to the elimination of many data instances, which ultimately leads to a significant loss of information that can otherwise use for training the classifier, this approach often produces less improved classification results. Hulse et al. proposed a repetitive undersampling technique to overcome this problem (Hulse, Khoshgoftaar, & Napolitano, 2009). The authors have investigated the performance of the major paradigms within repetitive undersampling, which generates an ensemble of models. Each repetitive technique used multiple subsets of original training data to build a model to reduce the number of information losses. However, repetitive sampling increases the complexity of the model.

Oversampling: oversampling is where some samples of the minority class replicate so that the number of samples is equal for all classes. This technique aims at balancing the majority and minority class distributions. The oversampling techniques have attracted researchers' attention due to their efficiency against the class imbalance problem. Random oversampling is an approach where random instances of the minority class replicate until the number of instances for each class becomes equal to the number of instances with the highest number of instances (Batista, Prati, & Monard, 2004). Oversampling has an advantage of no information loss over undersampling and often outperforms undersampling. However, its main drawback is the high possibility of overfitting due to replicated instances of the minority class. It happens because the approach creates more of the same data this study currently has without adding any diversity to the dataset. It is important to note that random oversampling is only applicable to the training set. The

reason behind it is that the test set must have originally corrected samples with no replicated entries and used replicated entries for testing results into false classification results. Apart from balancing the training set, random oversampling hardly adds any new information for enhancing classification performance since most of the instances in the minority classes only replicate the same information.

In 2002, Chawla et al. proposed the SMOTE algorithm based on oversampling technique (Chawla, Bowyer, Hall, & Kegelmeyer, 2002). SMOTE provides a solution for replicating the minority class instances using the combination of typical oversampling and random oversampling algorithms. It generates synthetic data samples by applying the concept of nearest neighbours. The KNN algorithm operates on the idea that instances belonging to the same class or instances with similar feature patterns are distributed close to one another in the data space. The SMOTE algorithm creates synthetic instances of the same class for each record belonging to the under-represented class by getting its K-nearest neighbours. Synthesized data instances locate at random positions between the instance in question and its neighbours. Like random oversampling, SMOTE results in all classes having the same sample size as the class with the highest number of instances. Although SMOTE is very useful and performs well for minority classes, it has a few disadvantages mentioned in SP-SMOTE. When choosing samples, there is some blindness, and because of the ignorance on the class distribution, it may increase the noise in the dataset. (ii) Because of creating fresh samples in a linear way, it is less effective to suppress the imbalance issue among the minority classes. (iii) It can motivate overfitting because when SMOTE creates fresh samples, they are possibly duplicated (Li, Wang, Li, Li, & Lan, 2021). SP-SMOTE overcome these disadvantages. However, the authors only experimented with binary class classification. Later on, because of the convenience and effectiveness of SMOTE, it was further studied by other researchers as bSMOTE: Borderline SMOTE (Hui, Wang, & Mao, 2005), SVM SMOTE: Support Vectors

Machine SMOTE (Nguyen, Cooper, & Kamei, 2011), ADASYAN: Adaptive Synthetic Sampling (He, Bai, Garcia, & Li, 2008). bSMOTE improves over SMOTE by oversampling minority class only at borderline samples. K-means clustering based on k-means smote was proposed by (Douzas, Bacao, & Last, 2018) based on SMOTE algorithm that ignores the creation of noise as well as successfully gets over imbalances within and between classes. ADASYAN applies the same oversampling principle as SMOTE. However, unlike SMOTE the number of instances generated for a given instance in the minority class is proportional to the number of neighbouring samples belonging to different classes. Thus, ADASYN tends to focus solely on outliers when generating new synthetic training examples. Learning adaptively and reducing bias is the main objective of ADASYN. However, both the bSMOTE and ADASYN approach tackle imbalanced datasets by producing new samples of minority class directly from existing samples. They hardly grasp the property of minority class samples. (Sáez, Krawczyk, & Woźniak, 2016) presented a paper where authors focused on examining the class characteristics for a multi-class imbalance dataset based on oversampling technique. They have tracked out subsets of each instance in every class and tightened the oversampling for each of them independently. By doing that, they could trace the information of each class component and boost more important and complex objects. Their experiment shows that oversampling can improve further by analyzing each instance's information. Whereas, in multi-class imbalanced classification, using standard oversampling does less consider on the important information of each instance. That means gathering important information from each example can significantly improve performance over the standard oversampling.

Few researchers started applying hybrid algorithms, such as combining the undersampling and oversampling approach to get better performance (Batista, Prati, & Monard, 2004; Batista, Bazzan, & Monard, Balancing training data for automated

annotation of keywords: a case study, 2003), where random oversampling used to replicate the minority class samples until they reach a balanced class distribution then, random undersampling was applied to remove the majority class to make distribution more balanced. A study assessed different kinds of techniques for imbalanced data where authors used machine learning algorithms to identify encrypted network traffic (Vu, Tra, & Nguyen, 2016). The authors have categorized each network flow by 22 statistical features. The study used cost function modification, undersampling, and oversampling approaches to prepare training data for the learning algorithms. Classification and Regression Trees (CART) and Random Forest (RF) algorithms showed the effectiveness of classification accuracy in that imbalanced data.

In 2009, Peng et al. proposed a data gravitation-based model for traffic classification using Newton's gravitation law (Peng, Yang, Chen, & Abraham, 2009). However, this proposed method performs less effective when applied to highly imbalanced datasets. After a couple of years, the authors proposed IDGC: an imbalanced data gravitation-based classification model to solve the internet traffic classification problem. They have found that their method performed very well (Peng, Zhang, Chen, & Yang, 2017). In recent research, authors have conducted an exploratory study on class imbalance and solution for network traffic classification where they experimented with latest methodologies and approaches for class imbalance problem (Gómez, Hernández-Callejo, Martínez, & J.Sánchez-Esguevillas, 2019).

2.5.2 Algorithm Level

At the algorithm level, one or more algorithm coordinates the calculation of an approach to tackle the issue of class imbalance in classification. To thrive the predicted accuracy of a model for the majority class, it combines some weak classifiers into a strong classifier utilizing several selection methods. The main purpose of this approach is to

improve the existing classification algorithms to make them suitable for imbalanced datasets. This approach also can be called as ensemble method. The method of selecting can divide into two types: hard ensemble and soft ensemble (Theofilatos, Chen, & Antoniou, 2019). It is one of the famous approach to tackle imbalanced classification problem (Błaszczyszki & Stefanowski, 2015; Galar M., Fernández A., Barrenechea E., Bustince H., & F., 2012; Krawczyk, Woźniak M., & G., 2014). Ensemble learning involves combining different methods such as Random Forest, Bagging, Boosting with resampling, or cost-sensitive methods as a hybrid approach. This approach takes place in the training phase. SMOTEBoost (Chawla, Lazarevic, Hall, & Bowyer, 2003) combines boosting with data sampling, where it leads to several highly skewed datasets to improve performance prediction of minority classes. A novel approach combined random undersampling with a boosting algorithm called RUSBoost, which proposed improving the classification model's performance on imbalanced data (Seiffert C., Khoshgoftaar T. M., Van Hulse J., & A., 2010). The authors claimed that RUSBoost is a faster, simpler, and less complex alternative to SMOTEBoost.

Recently a study combined resampling and ensemble algorithms called SmoteAdaNL (Liu, Ruoyu, & Ming, 2016). The resampling technique was applied to increase the amount of minority class flow in order to improve the classification accuracy of the minority class. Then ensemble learning algorithm increased the weight of misclassified flow samples. (Sun, et al., 2015) introduced a novel ensemble method that converted the imbalanced binary processing to multiple balanced learning. This method works without requiring to add minority instances or remove from majority instances to get rid of imbalanced data.

However, there is an absence of a proper understanding of the difference in imbalanced learning. Usually, most imbalanced ensemble techniques are computationally expensive

because of learning time and memory constraints. It is also challenging to measure correlation from different types of learners.

2.5.3 Cost-sensitive Level

Cost-sensitive learning (CSL) is an approach that indirectly modifies the imbalanced samples in datasets. It uses various cost metrics to find the cost of misclassifying the specific data sample. Mainly this approach is used to apply to multi-class imbalance problems. This learning approach aims to reduce the total cost, and it handles different misclassification differently.

Most recently, a study explored the class imbalance issue on a dataset where the authors (Mienye & Sun, 2021) analyzed the performance of cost-sensitive learning methods. They focused on thriving cost-sensitive classifiers by tuning the targeted function of a few well-recognized algorithms, namely, random forest, logistic regression, extreme gradient boosting and decision tree to predict the medical diagnosis appropriately. Since the resampling techniques are just the opposite of cost-sensitive learning, they hardly perform or modify anything on the distribution of original data. They initially employed the standard version of the mentioned algorithms to measure a performance comparison baseline. Then they developed their version of the cost-sensitive learning algorithm. They used four medical datasets for the experiment and performance comparison: chronic kidney disease datasets, Cervical Cancer Risk Factors, Pima Indians Diabetes, and Haberman Breast Cancer. The results showed that their developed cost-sensitive learning algorithm outperforms other standard algorithms.

Recently Feature selection approach has proved to be effective in the class imbalance problem. The feature selection method intends to lower the data thickness. Filters, wrappers, and hybrid methods are the three types of feature selection methods. Filter methods utilize a proxy measure to score a feature or a feature subset. Filter methods are

independent of the classification algorithm. Classical methods can apply in network traffic classification, such as information gain ratio and information gain. The experimental outcome demonstrated that these methods could significantly lower the thickness of traffic data without degrading traffic classification accuracy. These authors used a feature selection approach to address the class imbalance problem in network traffic classification (Zhang, Lu, Qassrawi, Zhang, & Yu, 2012). In their research, they have proposed two feature selections. One is WSU_AUC which combines Weighted Symmetrical Uncertainty and Area Under roc Curve to select features of the specific classifier. Another is SRSF which determines the Robust and Stable Features from the acquired outcomes by WSU_AUC. However, in their experiments, the true positive rate of each application gained with the robust WSU_AUC selected features was hardly higher constantly.

Misclassification costs may classify into two types: class-dependent and example-dependent. Class-dependent has attracted more attention because it occurs in real applications most of the time. In example-dependent cost, classifying different examples varies according to different classes (Zhou, 2010). A study focusing on class-dependent cost (Liu & Liu, 2012) proposed a cost metric called “weighted cost matrix (WCM)”. The study determines a reasonable weight for each cost of FCM (flow-based cost matrix), where authors proposed a metric to gauge the imbalance degree of the multi-class dataset. The authors tested the cost metric on twelve traffic traces in the experiment. A classification algorithm C4.5 decision tree uses to classify the traffic, and MetaCost uses for the cost-sensitive learning algorithm. The experiment's outcome shows that WCM increases the flow recall and byte recall by 3.5% and 11.6% over FCM. However, according to their investigation, byte recall is lower than flow recall which needs further improvement.

Previously, few researchers combined the data manipulation and classifier levels to address the class imbalance problem. A study (Thai-Nghe, Gantner, & Schmidt-Thieme, 2010) have presented two experimental methods which deal with class imbalance problems using the resampling technique and CSL. The first one combined various resampling techniques with CSL using SVM. Second one proposed using CSL by optimizing the cost matrix. The first one helps increase the classifier's performance, while the second one helps minimize the misclassification cost. The advantage of their experiment is that they hardly need to change the classifier internally. However, the experiment focused on binary classification, and the dataset was about various diseases which need to apply against an imbalanced network dataset.

Even though most of the studies mentioned here claim to acquire better classification results, the pros and cons of different algorithms depend on the features, nature, and distribution of traffic data in the dataset. Table 2.1 shows some key studies which achieve significant outcomes against the class imbalance problem in classification.

Table 2. 1: Previous key studies on class imbalance

Method	Key Studies	Algorithm	Type of Dataset Used	Type of Class Imbalance	Ref.
Data Level	A study of the behavior of several methods for balancing machine learning training data	SMOTE+ENN and SMOTE+TomekLinks	Disease	Binary-class	(Batista, Prati, & Monard, 2004)
	An empirical comparison of repetitive undersampling techniques	Under-sampling	Disease, Software Engineering	Binary-class	(Hulse, Khoshgoftaar, & Napolitano, 2009)
	Smote: synthetic minority over-sampling technique	SMOTE	Disease, Oil slick, Gas can, Satellite image, Pattern recognition	Binary-class	(Chawla, Bowyer, Hall, & Kegelmeyer, 2002)
	Borderline-SMOTE: A new over-sampling method in imbalanced data sets learning	Borderline-SMOTE1, Borderline-SMOTE2	Disease, Image	Binary-class	(Hui, Wang, & Mao, 2005)
	Borderline over-sampling for imbalanced data classification	SVM, Borderline-SMOTE	Disease	Binary-class	(Nguyen, Cooper, & Kamei, 2011)
	ADASYN: Adaptive synthetic sampling approach for imbalanced learning	ADASYN: Improved SMOTE	Disease	Binary-class	(He, Bai, Garcia, & Li, 2008)
	Learning from Imbalanced data for encrypted traffic identification problem	Random under-sampling, Random over-sampling	Network	Multi-class	(Vu, Tra, & Nguyen, 2016)
Algorithm Level (Ensemble)	SMOTEBoost:Improving prediction of the minority class in boosting	SMOTE+Standard bootsing	Intrusion detection, Satellite Image, Disease	Multi-class, Binary-class	(Chawla, Lazarevic, Hall, & Bowyer, 2003)
	A Hybrid Approach to Alleviating Class Imbalance	Random under-sampling + boosting	Disease	Binary-class	(Seiffert C., Khoshgoftaar T. M., Van Hulse J., & A., 2010)
	A novel ensemble method for classifying imbalanced data	Under-sampling, Over-sampling, ClusterBal, SplitBal	Disease	Binary-class	(Sun, et al., 2015)

Table 2. 2: Previous key studies on class imbalance (continued)

Method	Key Studies	Algorithm	Type of Dataset Used	Type of Class Imbalance	Ref.
Cost-sensitive level	Studying cost-sensitive learning for multi-class imbalance in internet traffic classification	WCM (Weighted Cost Metrix), C4.5 decision tree, MetaCost	Network	Multi-class	(Liu & Liu, 2012)
	Performance analysis of cost-sensitive learning methods with application to imbalanced medical data	Modifies objective function of existing algorithm, XGBoost	Disease	Binary-class	(Mienye & Sun, 2021)
	Cost-sensitive learning methods for imbalanced data	S-CSL, S-CSL, SVM	Disease	Binary-class	(Thai-Nghe, Gantner, & Schmidt-Thieme, 2010)

2.6 Conclusion

This chapter discussed current literature and efforts in the class imbalance problem. While the existing studies have demonstrated significant progress and achievements, there is still a gap to enhance the classification accuracy and efficiency of imbalanced class distribution. Handling class imbalance is very challenging, and it requires proper solutions without data loss and overfitting. As such, this study attempts to examine various data-level methods capable of improving the sensitivity of classification accuracy using imbalance techniques.

CHAPTER 3: METHODOLOGY

3.1 Introduction

The literature reviews from the previous chapter provide the theoretical base for this study. This chapter pursues the research philosophy and describes the instruments that form the research methodology in this study. This study focuses examining on how the various data balancing techniques affect the sensitivity of minority classes in multiclass classification. The experimental methodology utilized resampling techniques using machine learning algorithms.

3.2 Research Methodology

This section explains the research process and activities of this study. The general research methodology is divided into five main phases: 1) Literature Review 2) Problem formulation 3) Define the research design 4) Evaluation 5) Documentation. Figure 3.1 depicts the phases of the general research process in this study.

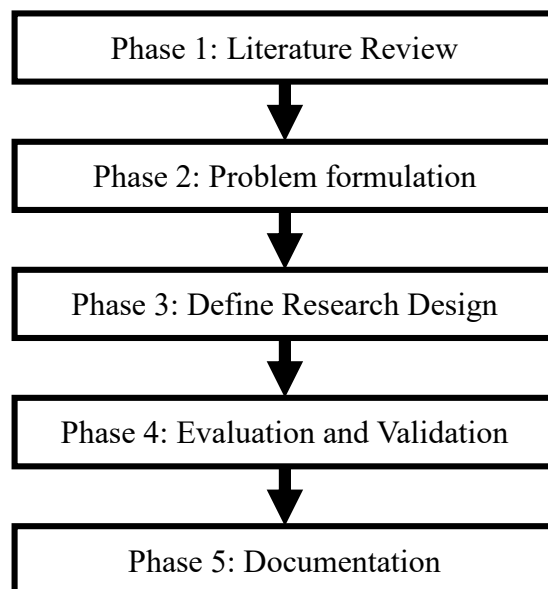


Figure 3. 1: Phases of the general research methodology

Phase 1 describes an extensive study on the class imbalance techniques in network traffic classification to get state of the art knowledge on this field. In phase 2 explores the gaps in the studies conducted by previous researchers and defines the problem in the research area, more specifically, the distribution of imbalanced classes. The next phase describes the research design after a thorough gap evaluation from the previous phases. In Phase 4, the classification results obtained from the imbalanced dataset are compared with a balanced dataset using various sampling techniques. Finally, phase 5 is writing a detailed report of this research.

The research process adopted a goal-oriented method using the SMART approach, which includes specific, Measurable, attainable, Relevant, and Time. The SMART method is useful because it highlights the specific deliverables of the research. Therefore, this method can help this research focus on what this study is doing and increase the chance of achieving our goal within the projected time frame. The SMART method describes the goals and the various components in detail shown in Figure 3.2.

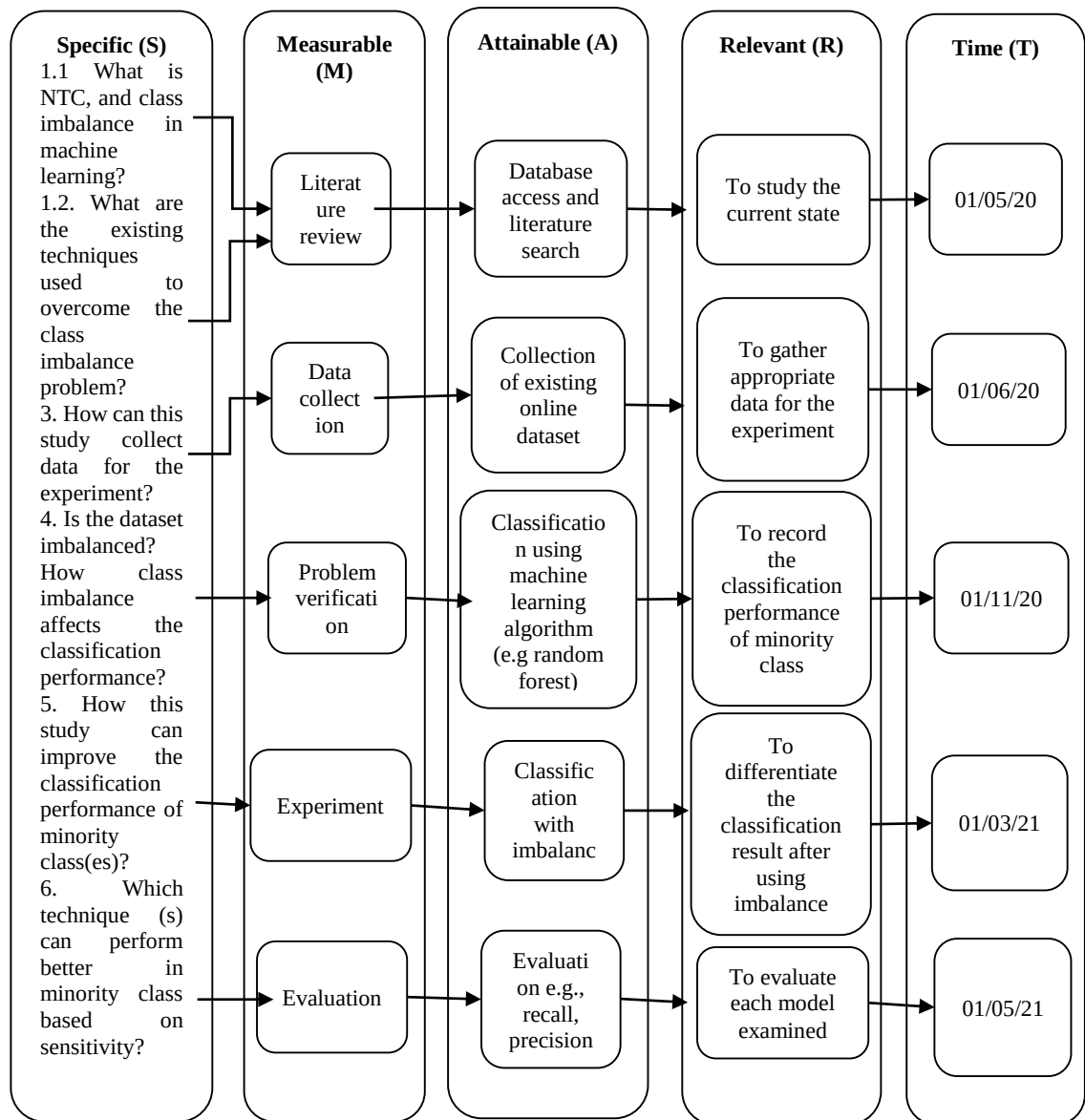


Figure 3. 2: SMART Method

This study answers all the questions using the smart method mentioned in specific goals by measuring the objectives.

1. What is NTC and class imbalance in machine learning? And What are the existing techniques used to overcome the class imbalance problem? This study gets the answers by studying the current state measured by literature review accessing research database.

2. How can this study collect data for the experiment? That can measure by data collection. This study finds existing online datasets to gather appropriate data for the experiment,
3. Is the dataset imbalanced? How class imbalance affects classification performance? Answering these questions can establish the research problem. The study uses a machine-learning classification algorithm to record the performance of minority classes.
4. How can this study improve the classification performance of minority class(es)? The experiment classification measures it with imbalance techniques used to differentiate the classification result after using imbalance techniques.
5. Which technique (s) can perform better in minority classes based on sensitivity? This study used metrics such as accuracy, recall, precision and f1-score to measure the performance evaluation.

3.3 Experimental Methodology

The experimental methodology illustrates the standard procedure while executing the process in the study. It divided the experimental methodology into four general stages: data collection, data processing, machine learning, and evaluation. Figure 3.3 summarizes the experiment flow in this study.

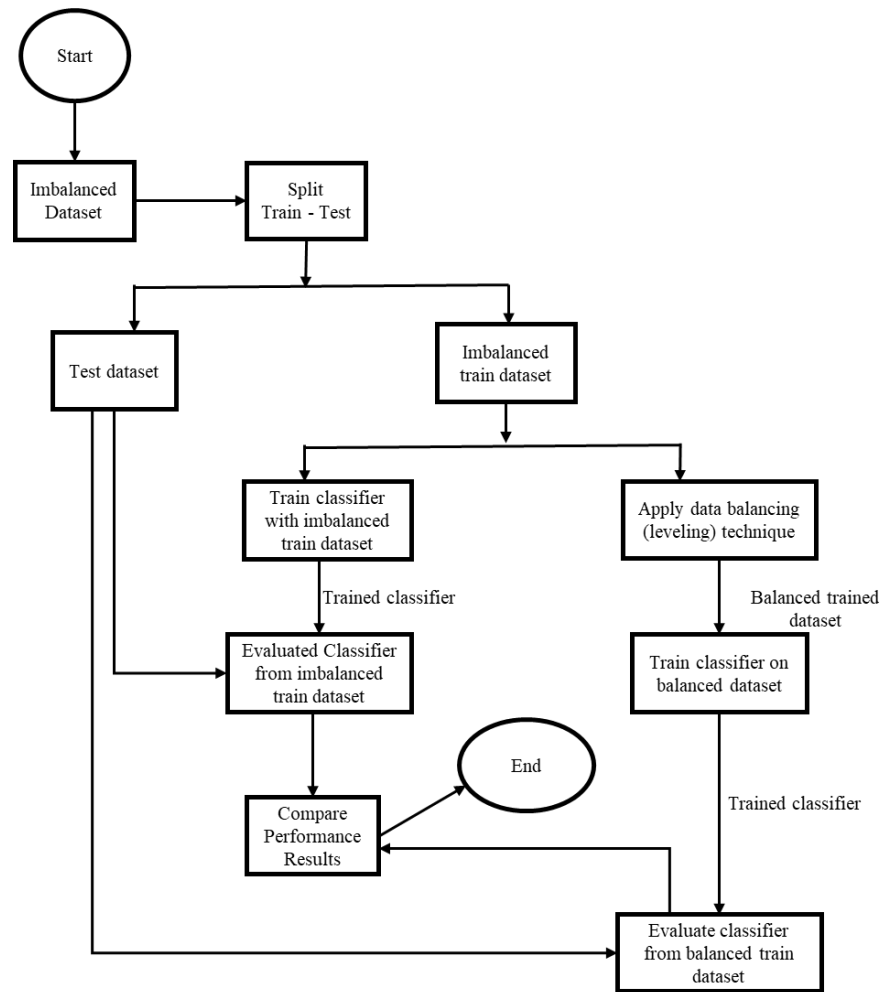


Figure 3. 3: Flow of experiment

3.3.1 Dataset

The essential aspect of any experiment is the dataset, specifically where machine learning is involved. Datasets are a saved collection of data that correspond to rows and columns in a table that contains all the variables and records of a specific task. This study examines an extensive and detailed publicly available dataset on the Kaggle website

(Rojas, 2017). The study collected data in a network section from Universidad Del Cauca, Popayán, Colombia, by performing packet captures at different times over six days and stored in a comma-separated value (CSV) file. This dataset has 87 features where each of the instances has the information of IP flow, including application name, type, port, source, destination, etc. The last column is the objective class of the dataset, and it holds the application name following the code number stored in the L7Protocol attribute. In comparison, most network traffic classification datasets aim only to identify the type of application an IP flow holds (WWW, DNS, FTP, P2P, Telnet, etc.). This dataset goes a step further by generating machine learning models capable of detecting specific applications such as WhatsApp, Facebook, Wikipedia, Office 365, Yahoo, YouTube, Skype, etc., from IP flow statistics of a total of 77 applications, as shown in Table 3.1.

Table 3. 1: List of applications in the dataset

<i>NO.</i>	<i>Application Name</i>	<i>NO.</i>	<i>Application Name</i>	<i>NO.</i>	<i>Application Name</i>	<i>NO.</i>	<i>Application Name</i>
1	Skype	21	ssh_no_cert	41	radius	61	apple_itunes
2	Whatsapp	22	mail_imaps	42	whois_das	62	Spotify
3	Telegram	23	Yahoo	43	99taxi	63	Netflix
4	msn	24	Gmail	44	easytaxi	64	Youtube
5	h323	25	Rtmp	45	Bgp	65	http_download
6	Teamspeak	26	google_maps	46	apple	66	http
7	Oscar	27	Waze	47	ip_icmp	67	http_connect
8	Dropbox	28	Cnn	48	Mqtt	68	http_proxy
9	ms_one_drive	29	Lastfm	49	Qq	69	Tor
10	Cloudflare	30	wikipedia	50	skinny	70	Starcraft
11	Ubuntuone	31	office_365	51	simet	71	ftp_data
12	apple_icloud	32	microsoft	52	edonkey	72	ftp_control
13	lotus_notes	33	windows_update	53	teamviewer	73	unencryped_jabber
14	Oracle	34	ms_sql	54	timmeu	74	Twitch
15	citrix_online	35	opensignal	55	Dns	75	Citrix
16	Ebay	36	ip_ospf	56	Upnp	76	bit_torrent
17	Amazon	37	Snmp	57	facebook	77	Socks
18	Google	38	Nfs	58	instagram		
19	Ssh	39	openvpn	59	twitter		
20	Ssl	40	Ntp	60	deezer		

3.3.2 Data Processing

After collecting the data, this study starts pre-processing the dataset. Pre-processing involved removing outliers and other irrelevant attributes. The dataset is in the form of a CSV (comma-separated value) file. Relevant statistical features are extracted from the packet headers. The applications are further divided into fifteen classes, namely; Chat 12119 (1.158%), Cloud 12088 (1.155%), Ecommerce 15840 (1.514%), Gaming 2 (0.0002%), Google 256726 (24.530%), Interactive 131535 (12.568%), Mail 17015 (1.626%), Maps 154 (0.01471%), Media 741 (0.0708%), Microsoft 32758 (3.130%), Multimedia 33 (0.00315%), Services 3425 (0.3270%), Social_media 13494 (1.289%), Streaming 47463 (4.535%), Web_browsing 503180 (48.079%) according to Table 3.2 below.

Table 3. 2: Categories of application in the dataset

Categories	Instances (percentage)	Applications
chat	12119 (1.158%)	skype, whatsapp, telegram, msn, h323, teamspeak, oscar.
cloud	12088 (1.155%)	dropbox, ms_one_drive, cloudflare, ubuntuone, apple_icloud, lotus_notes, citrix, citrix_online.
ecommerce	15840 (1.514%)	ebay, amazon
google	256726 (24.530%)	Google
interactive	131535 (12.568%)	ssh, ssl, ssh_no_cert
mail	17015 (1.626%)	mail_imaps, yahoo, gmail, rtmp
maps	154 (0.01471%)	google_maps, waze
media	741 (0.0708%)	cnn, lastfm, Wikipedia
microsoft	32758 (3.130%)	office_365, microsoft, windows_update
multimedia	33 (0.00315%)	twitch, opensignal
services	3425 (0.3270%)	ip_ospf, snmp, nfs, openvpn, ntp, radius, whois_das, 99taxi, easytaxi, bgp, apple, ip_icmp, mqtt, qq, skinny, simet, socks, teamviewer, timmeu, dns, upnp.

Table 3. 3: Categories of application in the dataset (continued)

Categories	Instances (percentage)	Applications
social_media	13494 (1.289%)	facebook, instagram, twitter
streaming	47463 (4.535%)	deezer, apple_itunes, spotify, netflix, youtube.
web_browsing	503180 (48.079%)	http_download, http, http_connect, http_proxy, tor.
Gaming	2 (0.0002%)	Starcraft

However, since the gaming class has only two traffic instances, it was considered an outlier and removed from the dataset. In addition, data processing cleaned the label column from the dataset since all instances belong to the benign class, which is completely irrelevant and hardly add any information pertinent for traffic classification. Furthermore, string columns that were neither categorical nor convertible into numeric datatypes like Flow ID, Source IP, Destination IP, and Timestamp also has removed from the dataset.

3.3.2.1 Creating training and test datasets

This research deployed the percentage of split approach to create training and test datasets appropriately. Training the dataset is particularly used to get a better understanding and prepare the classifier model to learn about the data. Here sklearn's random train_test_split module is used in Python 3.6 to create a training and test set. This research examines the performance of a popular machine learning algorithm, Random Forest classifier, prior to and following the use of various data levelling techniques. The study aimed at exploring the performance of each data levelling technique as far as improving detection of the minority classes is concerned. The dataset splits as 70:30 ratios for training and testing, respectively, for all experiments. However, it is worth noting that this ratio changes after applying the data levelling techniques since the levelling applies to only the training instances. The ratio changes because data levelling results in synthetic

instances, which can lead to wrong testing results if such instances include as part of the testing data instances. The testing must be on original dataset instances to avoid misleading performance results.

3.3.3 Machine Learning

This section gives a detailed overview of the machine learning algorithms used in this research. In general, most of the machine learning algorithms assume and expect a readymade dataset for the training process. However, it is continuous when it is regarding the network traffic dataset because the network traffic flows dynamically from different sources to different destinations. For classification, this research uses the Random Forest algorithm. It comprises a large number of individual decision trees which made it an ensemble. Researchers have been using Random Forest for decades. It is proven to achieve higher accuracy for both binary and multiclass classification. In general, ML algorithms assume that all the classes formed in the dataset are fair. However, in the real world classification, the dataset is highly imbalanced or skewed towards the majority class. This study uses several balancing techniques: random undersampling, edited nearest neighbour, random oversampling, synthetic minority oversampling, and adaptive synthetic oversampling. In addition, this study also uses two mixed samplings techniques to handle a highly imbalanced dataset. The first mixed sampling is the synthetic minority oversampling technique with the edited nearest neighbour. The second mixed sampling is the synthetic minority oversampling technique with Tomek links. After applying each balancing technique differently to the imbalanced dataset, this study again implements the random forest classifier for each balancing technique individually. In that way, this study can compare each algorithm performance before and after using the imbalance technique for random forest classifier.

3.3.4 Evaluation

Various metrics are available to use in machine learning model evaluation. Depending on the application, model, and dataset, different evaluation metrics apply to evaluate the algorithm's performance properly. How machine learning algorithms are assessed and compared is influenced by the metrics used. The popular evaluation metrics used in machine learning are classification accuracy, confusion matrix, mean absolute error log loss, area under curve etc. Some specific metrics are used for classification model evaluation, such as accuracy, precision, recall, f1 score, and specificity. Choosing the correct evaluation metric is important, especially regarding the imbalanced class distribution. Because sometimes some metrics can mislead the classification result in different scenarios, such as accuracy, only accuracy cannot measure a classification result properly. Because accuracy only can measure the correct prediction out of total prediction. For this reason, this study needs to use other relevant evaluation metrics to evaluate the result in a proper way. This section provides an insight into some metrics used in this research to evaluate performance and efficiency. Any of the examples from the classification can take one of the four categories, i.e. true positive, true negative, false positive, or false negative. In this study, the minority class belongs to positive as our focus is to improve the classification performance of minority classes. Hence, the minority class is labelled as positive, and the majority class is labelled as negative. The confusion matrix provides more insight into the performance of a predictive model and determines which class can predict correctly, which incorrectly, and what type of errors may find. The Confusion Matrix is one of the most simple and straightforward measures for evaluating the model's accuracy and correctness. It performs to solve classification problems when the output divides into two or more classes. In this type of confusion matrix, each cell in the table has a specific and well-understood name, summarized in Table 3.3 in relation to minority and majority class.

Table 3. 4: Confusion matrix for an example that is predicted positive or negative

	Predicted		
Actual		Positive (minority)	Negative (majority)
	Positive (minority)	TP	FN
	Negative (majority)	FP	TN

True positive (TP)

True positive declare the percentage of positive examples correctly predicted (i.e. correctly detect minority as a minority).

True negative (TN)

True negative presents the percentage of negative examples that are correctly predicted (i.e. correctly detect majority as a majority).

False positive (FP)

False positive presents the percentage of negative examples that are misclassified (i.e. wrongly detect minority as a minority).

False negative (FN)

False negative represents the percentage of positive examples that are misclassified (i.e. wrongly detect majority as a majority).

Network traffic data is more often imbalanced as the network flows dynamically. In this study, the dataset used is found imbalanced. Evaluating imbalanced data requires appropriate metrics for measuring performance, and it all relies on needs and the context of the problem that are attempting to address. As a result, we may desire to reduce the number of False Positives or False Negatives. For evaluating the results of this experiment, this study uses recall, precision, and f1-score apart from accuracy.

Accuracy: Accuracy is the most used threshold for classification. The accuracy of a classifier in the field of network monitoring is the fraction of rightly classified traffic instances to the total number of instances. Literally, it provides the number of correct predictions out of total predictions. Since a misclassification of a minority instance can be highly disastrous and can result in huge losses to a business ranging from financial to lose customer trust, it is vital to consider other performance metrics for network traffic classification in addition to accuracy. It is defined by the equation below:

$$Accuracy = \frac{TP + TN}{(TP + TN + FP + FN)}$$

Recall: Recall represents the value of how well a model predicts the true positive rate for minority classes. It is also known as sensitivity which calculates the true positive rate of a prediction model. This metric is very important in the classification model as it shows the efficiency of the performance in prediction. In other words, it explicitly gives the probability of correctly classifying the network traffic. The below equation defines the recall metrics:

$$Recall \text{ (or Sensitivity)} = \frac{TP}{(TP + FN)}$$

Precision: Precision represents the rate at which a properly classified minority (positive) class takes up all positive classes. It is also called a positive predictive rate. Precision is defined as:

$$Precision = \frac{TP}{(TP + FP)}$$

F1-score: The F1-score conveys the balance between recall and precision in relation to accuracy. The F1-score is defined as:

$$F1 = 2 * \frac{Precision * Recall}{Precision + Recall}$$

The focus of this study is to maximize the rate of true positive (recall). Recall often affects the precision rate, and precision often affects the recall rate.

“In imbalanced datasets, the goal is to improve recall without hurting precision. These goals, however, are often conflicting, since in order to increase the TP for the minority class, the number of FP is also often increased, resulting in reduced precision.” (He & Ma, 2013, p. 55).

3.4 Environmental Setup

For the experiment, this study used a 64 bit Ubuntu desktop on top of 7th Generation Intel® Core™ i5-7200U processors with CPU @2.50GHz and 16 GB RAM. The whole experimental program is written with Python 3.6, which can be free of charge and downloaded from <https://www.python.org/downloads/release/python-360/>. An open-source python machine learning library called *scikit-learn* is used that can be installed from its Github repository at <https://github.com/scikit-learn/scikit-learn>. This *scikit-learn* library is very popular and maintained by a few hundred users who are delivering help to researchers throughout the community. This study used the Jupyter Notebook compiler

to run the program that can be found for free on its official website at <https://jupyter.org/install>.

3.5 Conclusion

This chapter describes the general research methods employed for the entire research to achieve the research objectives. This study outlines the research method in five main phases 1) Literature review, 2) Problem formulation. 3) Define the research design. 4) Validation and evaluation 5) Documentation. Besides the general research methodology, this chapter concluded with the experimental methodology used in the experiment, such as data collection, data processing, machine learning, and evaluation.

CHAPTER 4: RESULTS AND DISCUSSIONS

4.1 Introduction

Following the experimental methodologies described in the previous chapter, this chapter shows the results of this study. Various performance measurements are described, along with interpretation for each technique. The discussion section includes the analysis to make the presented results clearer. Moreover, the study compares the performance to each other in order to present the better performance in terms of sensitivity of minority class. Firstly, this study described classification solely using random forest. After that, this study describes the results of classification using imbalance techniques. Finally, this study compared and discussed all the results.

4.2 Distribution of classes in the dataset

It is clearly seen from Figure 4.1 that network traffic is highly imbalanced, with almost half of the instances belonging to the web_browsing class. In contrast, classes like chat, cloud, e-commerce, mail, maps, media, multimedia, services, and social_media are extremely under-represented. Implying that for a single class such as maps, even if all instances are wrongly classified, there is still a possibility of registering a classification accuracy of over 99%. Furthermore, the four classes maps, media, multimedia, and services altogether make only 0.41592% of the entire dataset. Implying that even if all the instances in these four classes are wrongly classified, a classification accuracy of up to 99.58408% can be achieved if all the other instances are correctly classified.

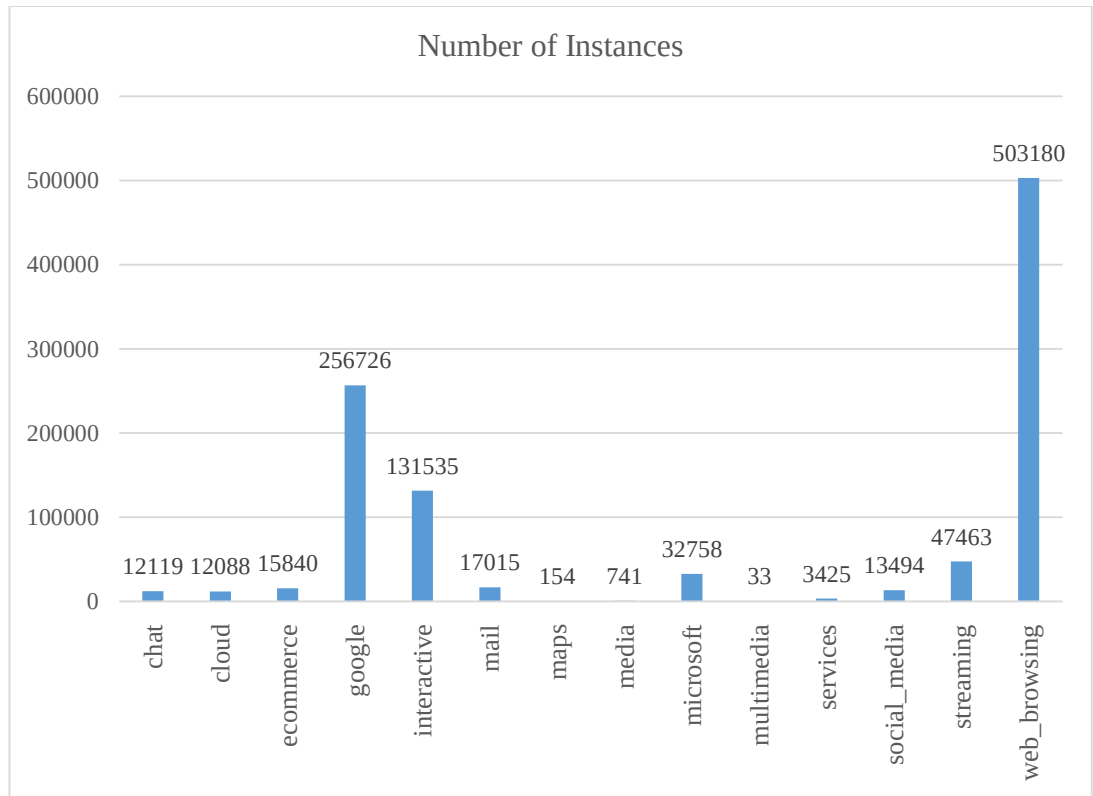


Figure 4. 1: Distribution of classes

4.3 Classification without Imbalance Technique

The study first trains the Random Forest classifier algorithm on the imbalanced dataset to compare later with the balanced dataset. Then it evaluates the performance using the metrics overall accuracy, data sensitivity, precision and f1-score. After applying the classifier algorithm, despite the high classification accuracy of above 98% for the classifier, recall shows that high accuracy can be misleading for minority classes like maps, media, and multimedia. The top three majority classes, such as google, interactive, and web_browsing, recorded 1.0 in terms of sensitivity. The below Figure 4.2 shows the sensitivity score of each minority class.

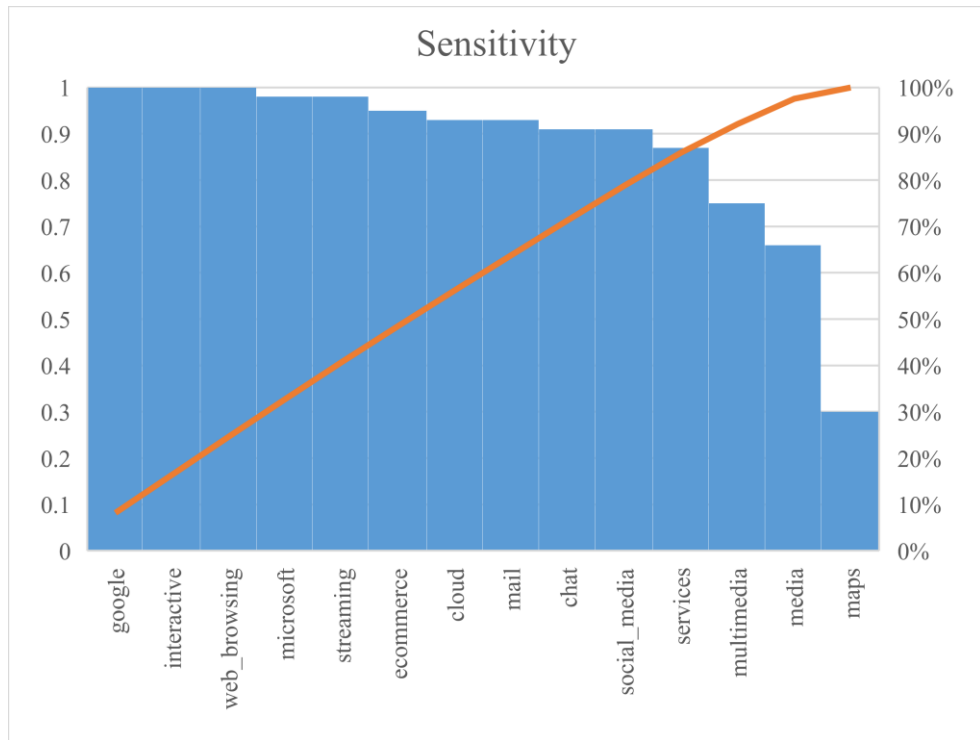


Figure 4. 2: Sensitivity of minority class in the dataset

Since shallow values are recorded with Random Forest recording as low as 0.30, 0.66, and 0.75 for maps, media, and multimedia, respectively, as shown in Figure 4.2 for sensitivity. Table 4.1 for overall accuracy, precision, and f1-score. Classification sensitivity of the Random Forest classifier is the benchmark of this experiment.

Table 4. 1: Performance of Random Forest classifier prior to balancing technique

Categories	Recall	Precision	F1-score	Overall Accuracy
chat	0.91	0.97		
cloud	0.93	0.98		
ecommerce	0.95	0.96		
google	1	0.99		
interactive	1	1		
mail	0.93	0.95		
maps	0.3	1	0.98	98.34
media	0.66	0.96		
microsoft	0.98	0.97		
multimedia	0.75	1		
services	0.87	0.96		
social_media	0.91	0.97		
streaming	0.98	0.97		
web_browsing	1	0.99		

4.4 Classification with Imbalance Techniques

This study mainly deployed two data-level approaches called undersampling and oversampling as well as their variants to deal with imbalanced data. For undersampling, this study deployed Random Undersampling and ENN. On the other hand, this study deployed Random Oversampling, SMOTE and Adasyn for oversampling. Also, this study mixed oversampling and undersampling together using SmoteENN and SmoteTomek algorithms. Each approach has different and unique characteristics to deal with class imbalance.

4.4.1 Random Undersampling

The multimedia class contains the least number of examples in the examining dataset, and only 33 instances belong to the multimedia class. That implies that applying random undersampling would result in 33 instances for each of the 14 classes in this dataset. After the application of the random undersampling technique, it can also be clearly seen that classes like *google*, *interactive*, and *web_browsing* lose a very large number of instances which significantly lowers the overall classification performance. The only improvement noticed in the three most minority classes where maps heighten the sensitivity from 0.3 to 0.59, media sensitivity heightens from 0.66 to 0.69, and multimedia heighten its sensitivity from 0.75 to 0.83. Even though the study recorded better classification results for the minority classes such as maps, media, and multimedia, compared with the classification results from the imbalanced class, the overall performance comprising precision and recall significantly falls for most of the classes, as depicted in Table 4.2.

Table 4. 2: Classification performance post deploying the random undersampling technique

Categories	Recall	Precision	F1-score	Overall Accuracy
chat	0.26	0.11		
cloud	0.58	0.24		
ecommerce	0.63	0.56		
google	0.37	0.62		
interactive	0.43	0.73		
mail	0.46	0.17		
maps	0.59	0.0	0.44	43.99
media	0.69	0.07		
microsoft	0.53	0.84		
multimedia	0.83	0.01		
services	0.55	0.09		
social_media	0.47	0.20		
streaming	0.33	0.40		
web_browsing	0.50	0.77		

However, the elimination of many data instances ultimately leads the dataset to a serious loss of information that the classifier would otherwise use to train. The approach hardly produces improved classification results. The f1-score falls from 0.98 to 0.44, less than fifty per cent from the imbalanced classification. Almost fifty per cent of the classes degrades half of its sensitivity. Figure 4.3 shows the sensitivity of the classification before and after deploying the random undersampling technique.

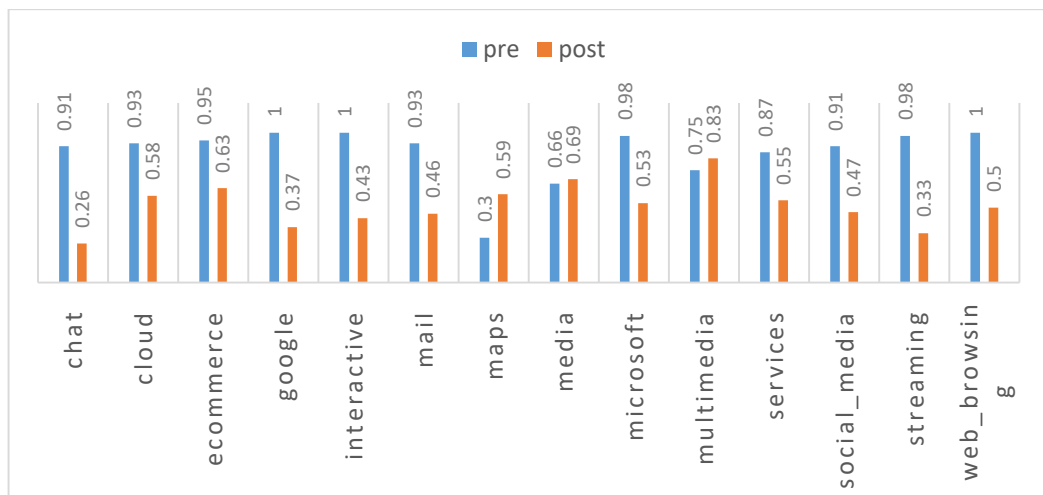


Figure 4. 3: Comparison of classification sensitivity pre and post Random Undersampling

In addition, the overall classification accuracy falls from 98.34% to 43.99%. Therefore, this implies that random undersampling is less suitable for deployment in a situation where the minority class contains a minimal number of examples as compared to the majority class.

4.4.2 Edited Nearest Neighbor (ENN)

ENN is a kind of undersampling technique that finds and removes the obscure and noisy examples by its k=3 nearest neighbour to the dataset. In this experiment, ENN hardly performs well except improving the sensitivity 0.75% to 0.83% of multimedia class shown in Figure 4.4. That is the only improvement so far this study finds from the ENN approach. Almost all classes lose the precision of classification, and the only class maps remain the same 1:1 as imbalanced classification. Although ENN decreased performance, it performed way better than the random undersampling technique. Other than that, it degraded overall accuracy from 98.34 to 88.51, f1 score from 0.98 to 0.89, and other individual classes. Table 4.3 shows the results of each evaluation metric for ENN.

Table 4. 3: Classification performance post deploying ENN

Categories	Recall	Precision	F1-score	Overall Accuracy
chat	0.37	0.87		
cloud	0.61	0.9		
ecommerce	0.66	0.76		
google	0.97	0.84		
interactive	0.99	0.96		
mail	0.38	0.87		
maps	0.11	1	0.89	88.51
media	0.07	0.89		
microsoft	0.86	0.84		
multimedia	0.83	0.48		
services	0.58	0.91		
social_media	0.61	0.92		
streaming	0.77	0.84		
web_browsing	0.99	0.91		

Figure 4.4 compares classification sensitivity for pre and post-application of ENN. According to the figure, the study finds that only three classes sensitivity: google,

interactive and web_browsing, reaches near to the benchmark. The classes chat, mail, maps, and media losses a significant degradation on sensitivity respectively, from 0.91 to 0.37, 0.93 to 0.38 to 0.11 and 0.66 to 0.07 as compared to pre and post application of ENN.

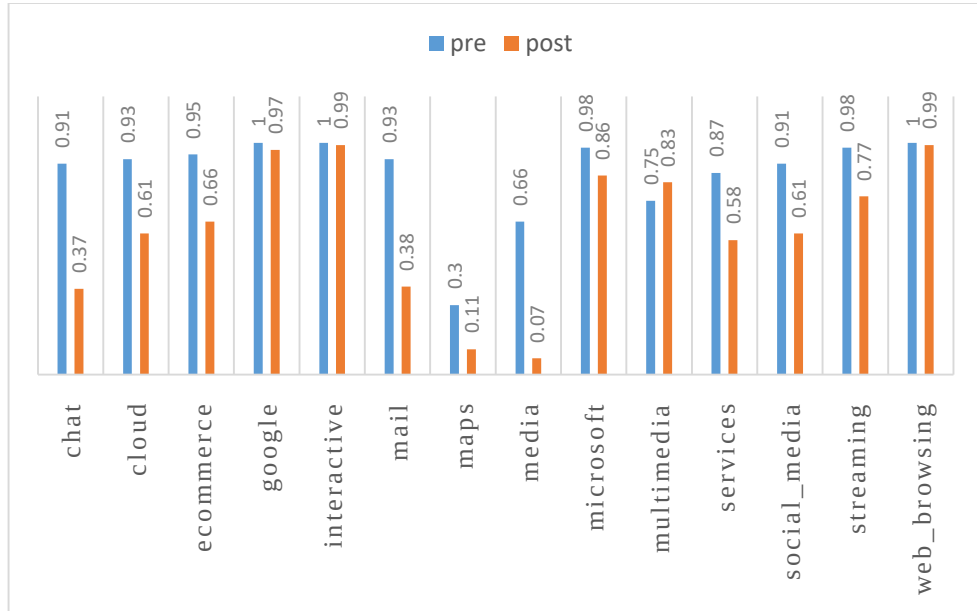


Figure 4. 4: Comparison of classification sensitivity pre and post ENN

4.4.3 Random Oversampling

After the application of oversampling, each class in training ends up with 352,066 instances. That was the size of the web_browsing class in the training set before the application of random oversampling. After the application of the random oversampling technique, the training set size increased from 701,202 to 4,722,774 instances. It increases the number of training examples in cases where the training data set is very large. The random oversampling approach has the disadvantage of increasing system resource requirements like processing time and memory. This study found the improvement of data sensitivity using the random oversampling technique for every minority class except two classes which remain unchanged such as chat and maps, compared to the benchmark. F1 score increased from 0.98 to 0.99. Precisions of minority classes such as cloud, e-

commerce, media, service, and social media also show significant improvement using this technique. Apart from other metrics, the overall classification accuracy improves slightly from 98.34% to 98.66%, as shown in Table 4.4.

Table 4. 4: Classification performance post deploying Random Oversampling technique.

Categories	Recall	Precision	F1-score	Overall Accuracy
chat	0.91	0.98		
cloud	0.95	0.99		
ecommerce	0.99	0.99		
google	1	0.99		
interactive	1	0.99		
mail	0.94	0.95		
maps	0.3	0.94	0.99	98.66
media	0.95	0.98		
microsoft	1	0.99		
multimedia	0.83	1		
services	0.93	0.98		
social_media	0.93	0.98		
streaming	0.98	0.99		
web_browsing	0.99	0.99		

Improvement in sensitivity of the minority classes like cloud, e-commerce, mail, media, multimedia, services, and social media is registered compared to the classification performance before the data balancing technique shown in Figure 4.5. Only two minority classes: chat and maps, remains unchanged.

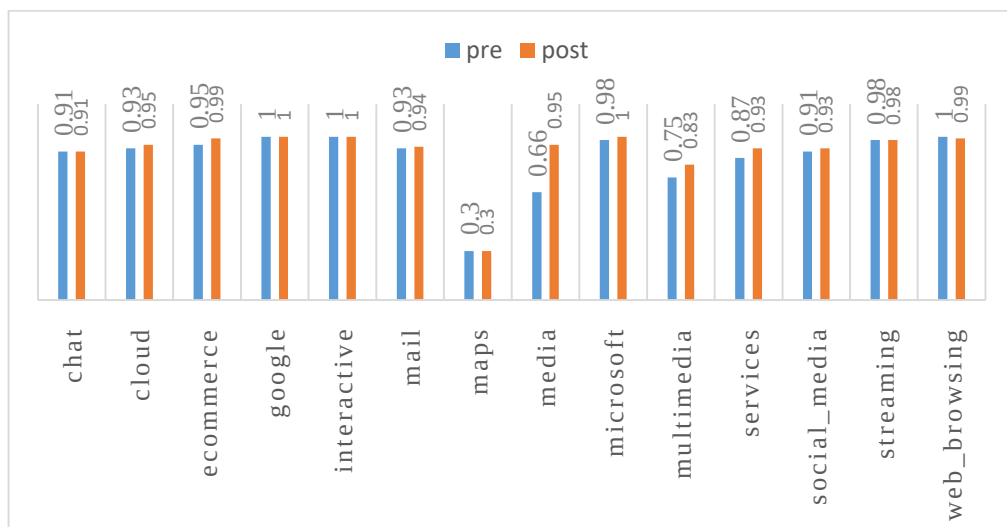


Figure 4. 5: Comparison of classification sensitivity pre and post Random Oversampling

4.4.4 Synthetic Minority Oversampling Technique (SMOTE)

SMOTE algorithm is a solution to minority class instances replication of the random oversampling algorithm. It generates synthetic data samples by applying the concept of nearest neighbours. The SMOTE algorithm creates synthetic instances of the same class for each record belonging to the under-represented class by getting its K-nearest neighbours. Synthesized data instances are located at random positions along with the lines between the instances in question and their neighbours. Like random oversampling, SMOTE results in all classes having the same sample size as the class with the highest number of instances. In this experiment, the application of SMOTE up-samples all minority classes to 337,341 instances increasing the training size to 4,722,774 instances. Data sensitivity performance for minority classes such as cloud, e-commerce, media, multimedia, service, and social media has improved. Few classes such as chat, mail, maps remain unchanged compared to the benchmark. The precision slightly decreases in some classes such as chat, cloud, interactive, mail, maps and services. The overall classification accuracy slightly improves from 98.34% to 98.41%, and the f1 score remains unchanged as 0.98, as shown in Table 4.5.

Table 4. 5: Classification performance post deploying SMOTE

Categories	Recall	Precision	F1-score	Overall Accuracy
chat	0.91	0.93		
cloud	0.94	0.97		
ecommerce	0.99	0.99		
google	0.99	0.99		
interactive	1	0.99		
mail	0.93	0.93		
maps	0.3	0.81	0.98	98.41
media	0.95	0.97		
microsoft	0.99	0.98		
multimedia	0.83	1		
services	0.94	0.95		
social_media	0.93	0.97		
streaming	0.98	0.99		
web_browsing	0.99	0.99		

Using SMOTE, this study finds there is no class where it degrades the sensitivity of the classification result. The sensitivity of the minority classes enhancement can be seen in the classification results in Figure 4.6 using a comparison between pre and post-application of SMOTE algorithm.

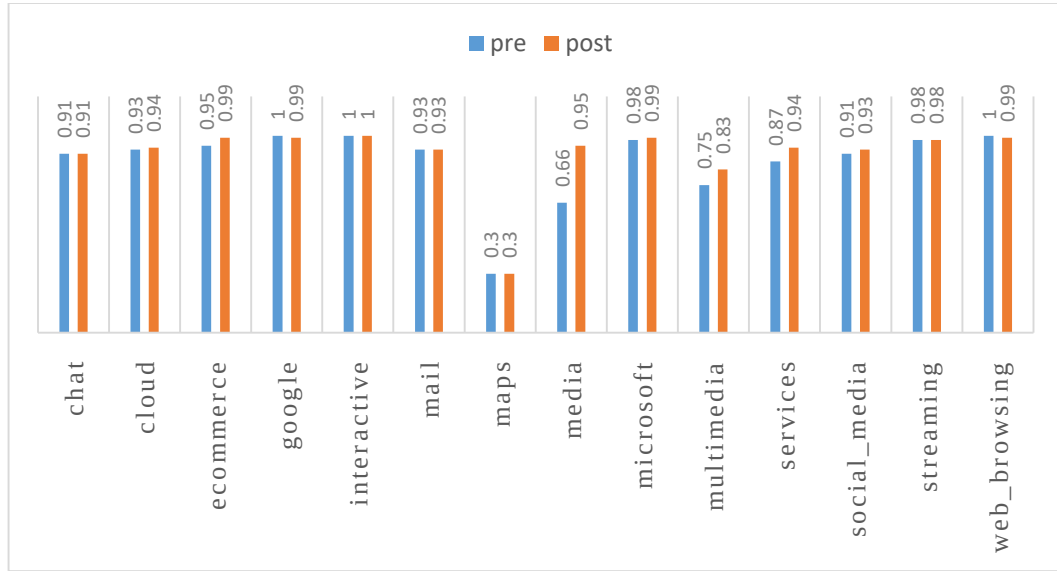


Figure 4. 6: Comparison of classification sensitivity pre and post SMOTE

4.4.5 Adaptive Synthetic Sampling (ADASYN)

ADASYN is mostly like SMOTE. The main difference is that ADASYN generates the number of synthetic samples only for individual original minority samples. After the application of ADASYN, the training size becomes 4,893,820 from the total train size. If this study compares with other oversampling techniques used in this research, such as random oversampling and SMOTE, Adasyn performs quite less. It degrades sensitivity for minority classes such as chat, mail, multimedia, services, and social media. The overall classification accuracy falls slightly from 98.34 to 98.26. Precision remained the same in two minority classes, namely cloud, multimedia and degraded the other classes. The study notices a significant improvement in precision for ecommerce minority class. The f1-score remains unchanged as before applying the imbalance technique at 0.98.

Table 4.6 presents the classification performance for all evaluation metrics used in this research for each class after deploying ADASYN.

Table 4. 6: Classification performance post deploying ADASYN

Categories	Recall	Precision	F1-score	Overall Accuracy
chat	0.89	0.96		
cloud	0.94	0.98		
ecommerce	0.95	0.98		
google	1	0.99		
interactive	1	1		
mail	0.92	0.94		
maps	0.35	0.88	0.98	98.26
media	0.69	0.97		
microsoft	0.99	0.97		
multimedia	0.62	1		
services	0.85	0.95		
social_media	0.9	0.96		
streaming	0.98	0.97		
web_browsing	1	0.98		

Adasyn increases the sensitivity for the least minority classes: maps and media, respectively, from 0.3 to 0.35 and 0.66 to 0.69. Other than that, sensitivity seems to improve on a majority class such as microsoft from 0.98 to 0.99. Adasyn algorithm degrades the performance for most classes compared to the classification result before applying the imbalance technique, as shown in Figure 4.7.

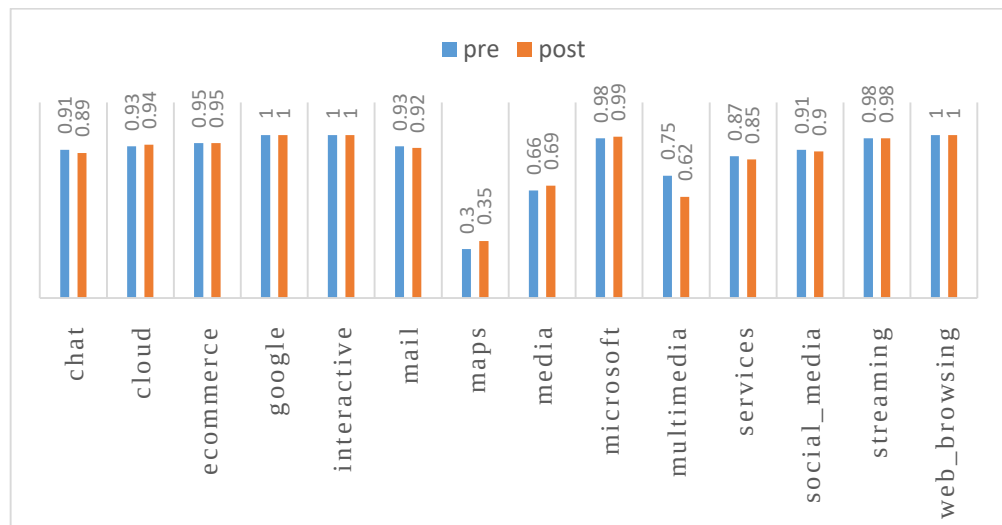


Figure 4. 7: Comparison of classification sensitivity pre and post ADASYN

4.4.6 Synthetic Minority Oversampling Technique with Edited Nearest Neighbor (SmoteENN)

SmoteENN combines oversampling and undersampling together. It can be called a hybrid sampling technique where SMOTE by defaults balance the data distribution, and ENN takes down the misclassified examples from all classes. SmoteENN shows better performance for individual minority classes. The precision falls significantly in three minority classes: chat, mail and maps, respectively from 0.97 to 0.35, 0.95 to 0.67 and 1 to 0.12. SmoteENN also degraded overall classification accuracy and f1-score respectively from 98.34 to 87.74 and 0.98 to 0.88. Table 4.7. presents the classification performance after deploying SmoteENN.

Table 4. 7: Classification performance post deploying SmoteENN

Categories	Recall	Precision	F1-sore	Overall Accuracy
chat	0.93	0.35		
cloud	0.94	0.82		
ecommerce	0.98	0.98		
google	0.83	0.94		
interactive	0.9	0.99		
mail	0.93	0.67		
maps	0.32	0.12	0.88	87.75
media	0.97	0.95		
microsoft	0.98	0.98		
multimedia	0.75	1		
services	0.95	0.8		
social_media	0.92	0.8		
streaming	0.94	0.79		
web_browsing	0.82	0.96		

The study neither record any performance degradation for SmoteENN in terms of data sensitivity. Only two minority classes remain unchanged such as mail and multimedia, as compared to the benchmark. Other than that, SmoteENN improves the sensitivity of minority classes, namely, chat, cloud, e-commerce, maps, media and service and social media, as shown in Figure 4.8.

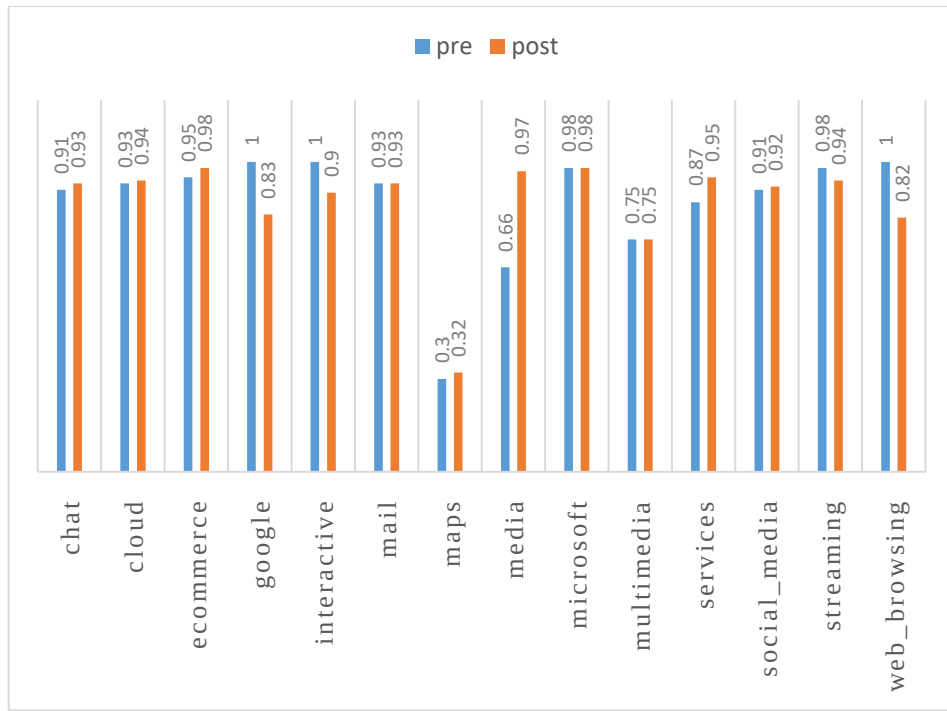


Figure 4. 8: Comparison of classification sensitivity pre and post SmoteENN

4.4.7 Synthetic Minority Oversampling Technique with Tomek Links (SmoteTomek)

Following SmoteENN, SmoteTomek is also a hybrid sampling approach that combines the oversampling technique (SMOTE) and undersampling technique (Tomek) to balance a dataset. Some noisy samples generate while training the dataset using SMOTE, and Tomeklinks removes the noisy samples from the dataset. After the application of SmoteTomek, this study observes better performance than any undersampling technique alone. Overall classification accuracy decreased slightly from 98.34 to 98.21, and the f1 score remains the same as the benchmarks' f1-score. Classification precision improves a little in ecommerce among the minority classes, whereas chat cloud, mail, maps, media seems to decrease the value. Although the overall classification accuracy has slightly degraded, this study can see the improvement of classification on minority classes. Compared to the imbalanced dataset, the average recall also improves significantly from

0.87% to 90%. Table 4.8 represents the evaluation metrics of the classification performance after deploying SmoteTomek in the dataset.

Table 4. 8: Classification performance post deploying SmoteTomek

Categories	Recall	Precision	F1-score	Overall Accuracy
chat	0.91	0.91		
cloud	0.95	0.97		
ecommerce	0.99	0.99		
google	0.99	0.99		
interactive	0.99	0.99		
mail	0.94	0.93		
maps	0.3	0.77	0.98	98.21
media	0.95	0.95		
microsoft	0.99	0.98		
multimedia	0.75	1		
services	0.94	0.95		
social_media	0.94	0.97		
streaming	0.98	0.98		
web_browsing	0.99	0.99		

SmoteTomek produces better results without degrading any of the minority class performance in terms of data sensitivity (recall). Minority classes such as cloud, e-commerce, mail, media, services, and social media significantly improve the sensitivity. This study also observed that three minority classes' sensitivity was unchanged as compared to the benchmark, as depicted in Figure 4.9.

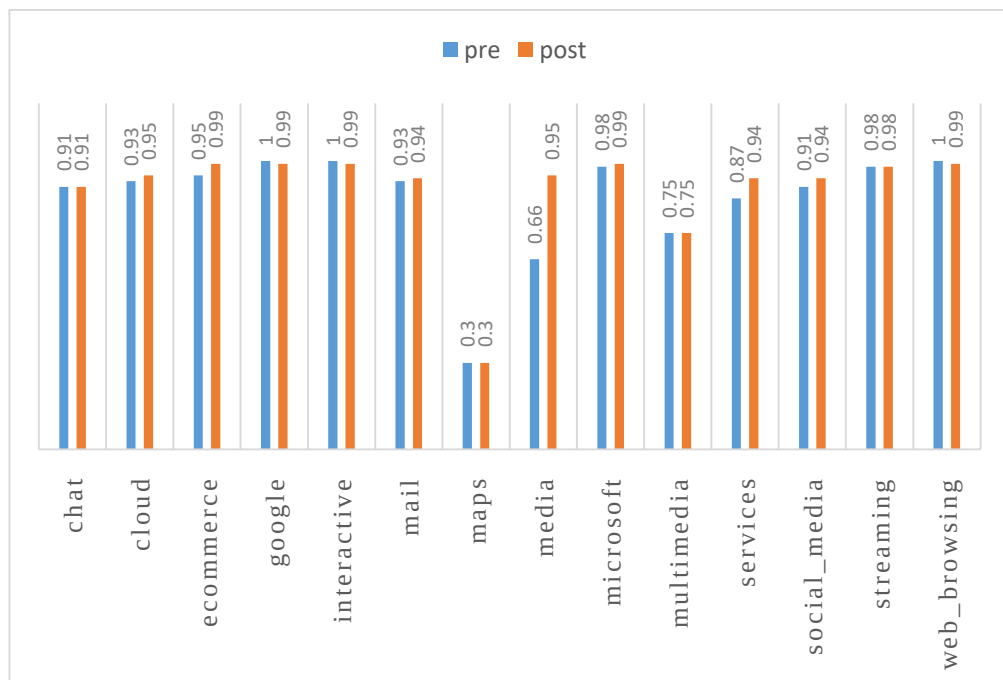


Figure 4. 9: Comparison of classification sensitivity pre and post SmoteTomek

As of now, this study has been trying to provide comparative results for each algorithm separately. Table 4.9 shows all the algorithms applied to the dataset to overcome the class imbalance in terms of classification sensitivity. In table 4.9, this study describes the performance of each algorithm in terms of sensitivity using a different colour such as brown for minority class, green for improved performance, red for decreased performance, and blue-grey for unchanged value.

Table 4. 9: Sensitivity of each algorithm applied

Categories	Imbalanced (benchmark)	RUS	ENN	ROS	Smote	AdaSyn	Smote ENN	Smote Tomek
chat	0.91	0.26	0.37	0.91	0.91	0.89	0.93	0.91
cloud	0.93	0.58	0.61	0.95	0.94	0.94	0.94	0.95
ecommerce	0.95	0.63	0.66	0.99	0.99	0.95	0.98	0.99
google	1	0.37	0.97	1	0.99	1	0.83	0.99
interactive	1	0.43	0.99	1	1	1	0.9	0.99
mail	0.93	0.46	0.38	0.94	0.93	0.92	0.93	0.94
maps	0.3	0.59	0.11	0.3	0.3	0.35	0.32	0.3
media	0.66	0.69	0.07	0.95	0.95	0.69	0.97	0.95
microsoft	0.98	0.53	0.86	1	0.99	0.99	0.98	0.99
multimedia	0.75	0.83	0.83	0.83	0.83	0.62	0.75	0.75
services	0.87	0.55	0.58	0.93	0.94	0.85	0.95	0.94
social_media	0.91	0.47	0.61	0.93	0.93	0.9	0.92	0.94
streaming	0.98	0.33	0.77	0.98	0.98	0.98	0.94	0.98
web_browsing	1	0.5	0.99	0.99	0.99	1	0.82	0.99

Minority classes Decreased Increased Unchanged

Figure 4.10 depicts the overall accuracy of each algorithm performed in this study. The major noticeable performance degrades are RUS, EMM and SmoteENN, where RUS degrades to 43.99, ENN degrades to 88.51, and SmoteENN degrades to 87.74 from the overall accuracy 98.34 before applying the imbalance technique. ROS and SMOTE algorithms show a little improvement from the benchmark. However, this study focuses

on improving the sensitivity of minority classes only as literature shows overall accuracy is less important in imbalanced classification.

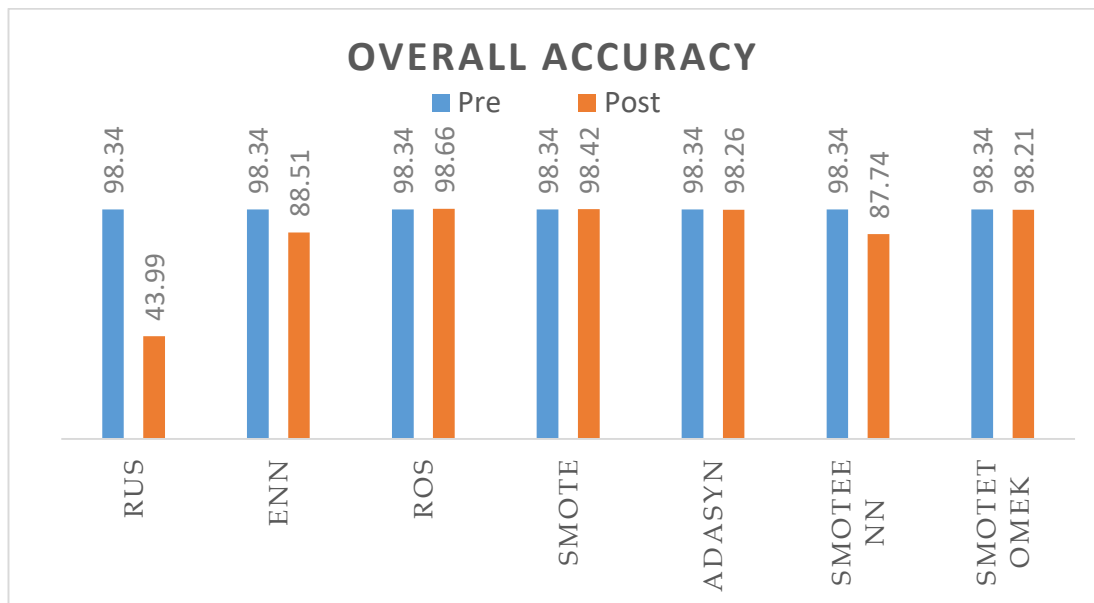


Figure 4. 10: Overall accuracy for pre and post application of each algorithm

Figure 4.11 compares the f1-score performance of all algorithms applied before and after applying the imbalance techniques. According to the figure, ROS produce a little improvement on the f1-score. The ENN and SmoteENN reduce f1-score slightly while RUS reduces a significant f1-score, and other classes such as SMOTE, Adasyn and SmoteTomek remains the same.

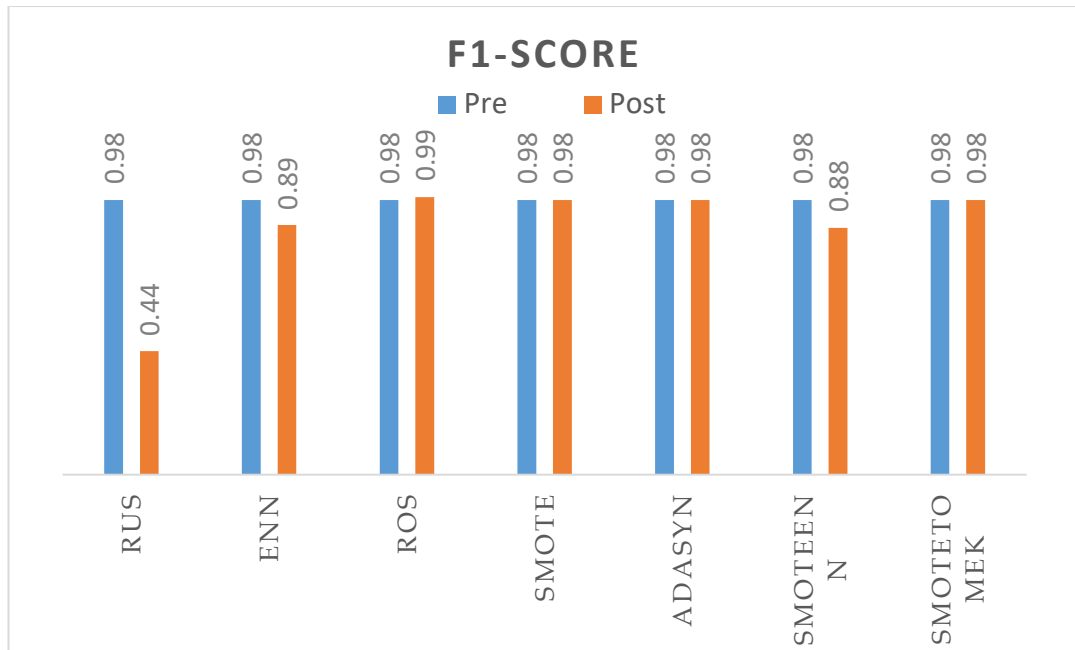


Figure 4. 11: F1-score for pre and post application of each algorithm

4.4 Conclusion

This chapter presented the experiments carried out in this research. The first experiment deployed a popular; Random Forest classifier for the classification of imbalanced data. Then the subsequent experiments trained the classifier for imbalanced data and evaluated the results with test data. After that, it compares all the performance of each algorithm with details. Among all the comparisons of each technique, algorithms that perform better are Random Oversampling, SMOTE, SmoteENN, SmoteTomek, as for the minority class is concerned. The efficiency of any imbalanced classification is relatively dependent on three major factors, i.e. the processing capacity of the hardware on which the system is installed, the choice of classifier, and the distribution of the instances in the dataset on which the classifier model is trained. However, it hardly scales with the number of traffic instances flowing through the system once the hardware is set. Therefore, it is crucial for a given hardware system to identify the most efficient sampling technique for the class imbalance problem. The performance of various data sampling techniques can be realized by training the classifier on both imbalanced and balanced

datasets that provide pertinent information for traffic classification. In addition, the training must conduct on a considerably comprehensive dataset that gives a good reflection of the state-of-the-art network traffic.

CHAPTER 5: CONCLUSION

5.1 Introduction

This chapter is the final chapter of the thesis. Based on the result from the experiment, it concludes this work by highlighting how the research objectives as mentioned in section 1.4 of this work are achieved, the contribution of the study, limitations, and a brief description of future work.

5.2 Achievement of research objectives

This study aimed at achieving three research objectives, as mentioned in section 1.4 and below for reference, which are:

- i. To study the impact of class imbalance in network traffic classification.
- ii. To investigate the mode of operation of the various techniques commonly used for data balancing in network traffic classification.
- iii. To evaluate the advantages and disadvantages of the data balancing model in terms of performance based on an imbalanced dataset.

The analysis conducted in chapter 4 suggests that the study achieves objective one by recording the ups and downs of each algorithm performance individually for each class on the imbalanced dataset. This study noted a significant impact of class imbalance on the dataset using those eight algorithms mentioned in table 4.9.

The study realizes the second objective by conducting a literature review of the related literature. The second objective highlighted a clear insight into the mode of operation of the various techniques commonly used for data balancing in network traffic classification, their characteristics, type of dataset, advantages, and weaknesses. The finding can significantly help other researchers and network administrators to determine which approach to apply under what circumstances.

Finally, the study realized the last objective by examining the performance evaluation of each technique carried out in chapter 4. This study compared every data balancing model applied to the imbalance dataset to get advantages and disadvantages thoroughly. In terms of data sensitivity, this study finds more advantages using oversampling techniques (ROS, SMOTE, and Adasyn) used in this research. This study recorded poor performance using two undersamplings (RUS and ENN) approaches. Using random undersampling and the edited nearest neighbour, most of the classes degraded their sensitivity compared to classification sensitivity using an imbalanced dataset.

5.3 Contribution of the research

This study has made the following major contributions.

- This study contributes by giving an explicit exploration as to what extent imbalance dataset or highly skewed dataset affects and plays a role in classifying network traffic using a machine learning algorithm.
- The study has proved that data balancing techniques can help the dataset to process efficiently and prepare for training the classifier to produce better classification results, especially on minority classes.
- The study can serve as a guide to the researcher who is interested in conducting experimental research on class imbalance issues in multi-class network traffic classification.

5.4 Limitations of the research

The discussion noted in previous chapters shows that this study has achieved its objectives. However, every study would definitely have some drawbacks or limitations due to certain constraints, so this study hardly goes beyond the exception as it also notes some limitations in the course of this conduct. Although this research produced an outstanding result towards the minority classes, it has three major limitations.

- a) **The number of datasets:** The benchmark was not the dataset in this study. The benchmark was the classification algorithm itself before using the imbalance technique. Although the dataset was not the benchmark in this research, this study used only one public dataset. Every dataset has different characteristics, size, distribution, number of examples, number of features or columns. This study could use several datasets to explore how the classification algorithm and imbalance techniques work against other datasets.
- b) **Lack of computing resources: This study needed a lot of computational resources to conduct the experiment, especially** during the training and generation process. Some of the algorithms take a long time to train the dataset and become frozen many times due to Random Access Memory (RAM) constraints. The 16 gigabyte (GB) RAM is less capable of generating synthetic data samples. Algorithms such as SMOTE, AdaSYN requires data generation feature to increase the number of samples in the minority class.
- c) **Only limited to data level techniques:** The study is limited to only data level techniques, among other techniques used to resolve class imbalance issues such as algorithm level and cost-sensitive level.

5.5 Future work

This research work has potential for future works. Few suggestions for future work outside the scope of this study identifies as follows:

Future work in this can think of conducting this research using multiple numbers of datasets with algorithm mentioned in this research. Collecting and capturing network traffic can be included in one of the future works to create a new dataset. Also, future work can consist of other techniques such as algorithm level, cost-sensitive level, or hybrid level like mixing with each technique. As the Random Undersampling and ENN

algorithms showed poor performance in this research, this study can include other undersampling algorithms such as Nearmiss1-3, One-Sided Selection (OSS), Neighborhood Cleaning Rule (NCR), Tomeklinks. Finally, regarding future work, a novel model can be proposed by modifying the SMOTE algorithm to increase the classifier's performance on minority classes.

5.6 Conclusion

This study is at the end of this thesis, where it shall provide a conclusion by mentioning the whole research very shortly. This research first discussed briefly network traffic classification and class imbalance issues. Then, this study presented some notable previous research works which tried to overcome the class imbalance problem. After that, this study discussed our dataset and research methodology towards the experiments of a multi-class classification where this study found the dataset was highly imbalanced towards the majority class. For the experiment, first, this study chooses the random forest machine learning classifier before using any imbalance techniques. Then the study utilizes various imbalance techniques to tackle the class imbalance problem in network traffic classification using the machine learning approach. Some techniques found better performance in terms of data sensitivity for individual minority classes but degraded overall classification accuracy, proving that overall accuracy is a less important evaluation metric for the purpose of imbalanced classification. Lastly, this study suggested some future work followed by the research limitation.

REFERENCES

- Ali, A., Shamsuddin, S. M., & Ralescu, A. L. (2015). Classification with class imbalance problem: a re-view. *International Journal of Advances in Soft Computing and its Applications*, 7(3), 176-204.
- An, H.-M., Lee, S.-K., Ham, J.-H., & Kim, M.-S. (2015). Traffic Identification Based on Applications using Statistical Signature Free from Abnormal TCP Behavior. *Journal of Information Science and Engineering*, 31(5), 1669-1692.
- Batista, G. E., Bazzan, A., & Monard, M. C. (2003). Balancing training data for automated annotation of keywords: a case study. *Proceedings of the Second Brazilian Workshop on Bioinformatics*, (pp. 35-43).
- Batista, G. E., Prati, R. C., & Monard, M. C. (2004). A study of the behavior of several methods for balancing machine learning training data. *SIGKDD Explor. Newsl.*, 6(1), 20-29. doi:10.1145/1007730.1007735
- Błaszczyszński, J., & Stefanowski, J. (2015). Neighbourhood sampling in bagging for imbalanced data. *Neurocomputing*, 150, 529–542. doi:10.1016/j.neucom.2014.07.064
- Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32. doi:10.1023/A:1010933404324
- Callado, A., Kamienski, C., Szabó, G., & al., e. (2019). A survey on internet traffic identification. *IEEE Commun. Surv. Tutor.*, 11(3), 37-52.
- Camacho, J., Padilla, P., Padilla, P., & Díaz-Verdejo, J. E. (2013). A generalizable dynamic flow pairing method for traffic classification. *Computer Networks*, 57, 2718-2732. doi:10.1016/j.comnet.2013.06.006
- Chang, W., Liu, Y., Xiao, Y., Yuan, X., Xu, X., Zhang, S., & Zhou, S. (2019). A Machine-Learning-Based Prediction Method for Hypertension Outcomes Based on Medical Data. *Diagnostics (Basel)*, 9(4), 178. doi:10.3390/diagnostics9040178
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). Smote: synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16(1), 321-357.
- Chawla, N. V., Lazarevic, A., Hall, L. O., & Bowyer, K. W. (2003). SMOTEBoost: Improving prediction of the minority class in boosting. *Proc. Principles Knowl. Discov. Databases* (pp. 107-119). Berlin, Heidelberg: Springer. doi:10.1007/978-3-540-39804-2_12

- Chen, W.-H., Hsu, S.-H., & Shen, H.-P. (2005). Application of SVM and ANN for intrusion detection. *Computers & Operations Research*, 32(10), 2617- 2634. doi:10.1016/j.cor.2004.03.019
- Dainotti, A., Pescapé, A., & Claffy, K. C. (2012). Issues and future directions in traffic classification. *IEEE Network*, 26(1), 35-40. doi:10.1109/MNET.2012.6135854
- Dong, B., & Wang, X. (2016). Comparison deep learning method to traditional methods using for network intrusion detection. *2016 8th IEEE International Conference on Communication Software and Networks (ICCSN)*, (pp. 581-585). Beijing. doi:10.1109/ICCSN.2016.7586590
- Douzas, G., Bacao, F., & Last, F. (2018). Improving imbalanced learning through a heuristic oversampling method based on k-means and SMOTE. *Information Sciences*, 465, 1-20. doi:10.1016/j.ins.2018.06.056
- Elazmeh W., Japkowicz N., & S., M. (2006). Evaluating misclassifications in imbalanced data. *Proc.of the 17th European Conference on Machine Learning*, (pp. 126-137).
- Erman, J., Mahanti, A., & Arlitt, M. (2007). Byte me: a case for byte accuracy in traffic classification. *Proc. of the 3rd Annual ACM Workshop on Mining Network Data*, (pp. 35-38). doi:10.1145/1269880.1269890
- Exposito, E., Gineste, M., Baudoin, C., & Aguilar, J. (2019). Towards the Deployment of Machine Learning Solutions in Network Traffic Classification: A Systematic Survey. *IEEE Communications Surveys & Tutorials*, 21(2), 1988-2014. doi:10.1109/COMST.2018.2883147
- Fernández, A., García, S., Galar, M., Prati, R. C., Krawczyk, B., & Herrera, F. (2018). *Learning from Imbalanced Data Sets*. Springer International Publishing. doi:10.1007/978-3-319-98074-4
- Finsterebusch, M., Richter, C., Rocha, E., Muller, J.-A., & Hanssger, K. (2014). A Survey of Payload-Based Traffic Classification Approaches. *IEEE Communications Surveys & Tutorials*, 16(2), 1135-1156. doi:10.1109/SURV.2013.100613.00161
- Galar M., Fernández A., Barrenechea E., Bustince H., & F., H. (2012). A review on ensembles for the class imbalance problem: bagging-, boosting-, and hybrid-based approaches. *IEEE Transactions on Systems, Man, and Cybernetics*, 42(4), 463–484.
- Gómez, S. E., Martínez, B. C., Sánchez-Esguevillas, A. J., & Callejo, L. H. (2017). Ensemble network traffic classification: Algorithm comparison and novel ensemble scheme proposal. *Computer Networks*, 127, 68-80. doi:10.1016/j.comnet.2017.07.018

- Gómez, S., Hernández-Callejo, L., Martínez, B., & J.Sánchez-Esguevillas, A. (2019). Exploratory study on Class Imbalance and solutions for Network Traffic Classification. *Neurocomputing*, 343, 100-119. doi:10.1016/j.neucom.2018.07.091
- Goss, R. G., & Nitschke, G. S. (Eds.). (2013). Automated network application classification: A competitive learning approach. *2013 IEEE Symposium on Computational Intelligence for Communication Systems and Networks (CICOMMS)* (pp. 45-52). Singapore: IEEE. doi:10.1109/CICOMMS.2013.6582853
- Harzevili, N. S., & Alizadeh, S. H. (2018). Mixture of latent multinomial naive Bayes classifier. *Applied Soft Computing*, 69, 516-527. doi:10.1016/j.asoc.2018.04.020
- He, H., & Ma, Y. (2013). *Imbalanced Learning: Foundations, Algorithms, and Applications*. Wiley-IEEE Press.
- He, H., Bai, Y., Garcia, E. A., & Li, S. (2008). ADASYN: Adaptive synthetic sampling approach for imbalanced learning. *Proceedings of the 5th IEEE International Joint Conference on Neural Networks*, (pp. 1322-1328). doi:10.1109/IJCNN.2008.4633969
- Hinton, G. E., Osindero, S., & Teh, Y.-W. (2006, July). A fast learning algorithm for deep belief nets. *Neural Computation*, 18(7), 1527-1554. doi:10.1162/neco.2006.18.7.1527
- Hui, H., Wang, W.-Y., & Mao, B.-H. (2005). Borderline-SMOTE: A new over-sampling method in imbalanced data sets learning. *Proceedings of the 1st International Conference on Intelligent Computing*, (pp. 878-887).
- Hulse, J. V., Khoshgoftaar, T. M., & Napolitano, A. (2009). An empirical comparison of repetitive undersampling techniques. *International Conference on Information Reuse & Integration* (pp. 29-34). Las Vegas: IEEE. doi:10.1109/IRI.2009.5211614
- IV, W. C., & Krawczyk, B. (2021). Multi-class imbalanced big data classification on Spark. *Knowledge-Based Systems*, 212. doi:10.1016/j.knosys.2020.106598
- Jie Yang, Yixuan Wang, Chao Dong, & Cheng, G. (2012). The evaluation measure study in network traffic multi-class classification based on auc. *International Conference on ICT Convergence (ICTC)*, (pp. 362-367).
- Khalife, J., Hajjar, A., & Diaz-Verdejo, J. (2014). A multilevel taxonomy and requirements for an optimal traffic classification model. *Int. J. Netw. Manag.*, 24, 101-120. doi:10.1002/nem.1855

- Krawczyk, B., Woźniak M., & G., S. (2014). Cost-sensitive decision tree ensembles for effective imbalanced classification. *Applied Soft Computing, 14 Part C*, 554-562. doi:10.1016/j.asoc.2013.08.014
- Kumar, R., Venkanna, U., & Tiwari, V. (2020). A Binary Classification Approach for Time Granular Traffic in SDWMN based IoT Networks. (pp. 531-534). Bengaluru: IEEE. doi:10.1109/COMSNETS48256.2020.9027336
- Kurt, B., Cemgil, A. T., Mungan, M., Polat, N., Özdoğan, A., & Saygun, E. (2012). Network management without payload inspection: Application classification via statistical analysis of bulk flow data. *2012 Future Network & Mobile Summit (FutureNetw)* (pp. 1-8). Berlin: IEEE.
- Kwon, J., Jung, D., & Park, H. (2020). Traffic Data Classification using Machine Learning Algorithms in SDN Networks. *2020 International Conference on Information and Communication Technology Convergence (ICTC)* (pp. 1031-1033). Jeju: IEEE. doi:10.1109/ICTC49870.2020.9289174
- Kwon, J., Lee, J., Yu, M., & Park, H. (2020). Automatic Classification of Network Traffic Data based on Deep Learning in ONOS Platform. *2020 International Conference on Information and Communication Technology Convergence (ICTC)* (pp. 1028-1030). Jeju: IEEE. doi:10.1109/ICTC49870.2020.9289257
- Labayen, V., Magaña, E., Morató, D., & Izal, M. (2020). Online classification of user activities using machine learning on network traffic. *Computer Networks, 181*. doi:10.1016/j.comnet.2020.107557
- Li, L., Zhang, H., Peng, H., & Yang, Y. (2018). Nearest neighbors based density peaks approach to intrusion detection. *Chaos, Solitons & Fractals, 110*, 33-40. doi:10.1016/j.chaos.2018.03.010
- Li, Y., Wang, Y., Li, T., Li, B., & Lan, X. (2021). SP-SMOTE: A novel space partitioning based synthetic minority oversampling technique. *Knowledge-Based Systems, 228*. doi:10.1016/j.knosys.2021.107269
- Liu Q, Liu Z, & M., H. (2010). Study on Internet traffic classification using machine learning. *Computer Science, 37*(12), 35-39.
- Liu, Z., & Liu, Q. (2012). Studying cost-sensitive learning for multi-class imbalance in internet traffic classification. *The Journal of China Universities of Posts and Telecommunications, 19*(6), 63-72. doi:10.1016/S1005-8885(11)60319-1
- Liu, Z., Ruoyu, W., & Ming, T. (2016). SmoteAdaNL: a learning method for network traffic classification. *Journal of Ambient Intelligence and Humanized Computing, 7*, 121-130. doi:10.1007/s12652-015-0310-y

- Liu, Z., Wang, R., Tao, M., & Cai, X. (2015). A class-oriented feature selection approach for multi- class imbalanced network traffic datasets based on local and global metrics fusion. *Neurocomputing*, 168, 365–381. doi:10.1016/j.neucom.2015.05.089
- Maheshwari, S., Jain, D. R., & Jadon, D. S. (2017). A Review on Class Imbalance Problem: Analysis and Potential. *IJCSI International Journal of Computer Science Issues*, 14(6), 43-51.
- Mienye, I. D., & Sun, Y. (2021). Performance analysis of cost-sensitive learning methods with application to imbalanced medical data. *Informatics in Medicine Unlocked*, 21. doi:10.1016/j.imu.2021.100690
- Mohana, R. M., Reddy, C. K., Anisha, P., & Murthy, B. R. (2021). Random forest algorithms for the classification of tree-based ensemble. *Materials Today*. doi:10.1016/j.matpr.2021.01.788
- Moore, A., Zuev, D., & Crogan, M. (2005). Discriminators for use in flow-based classification.
- More, A. (2016). Survey of resampling techniques for improving classification performance in unbalanced datasets.
- Mukherjee, S., & Sharma, N. (2012). Intrusion Detection using Naive Bayes Classifier with Feature Reduction. *Procedia Technology*, 4, 119-128. doi:10.1016/j.protcy.2012.05.017
- Namdev, N., Agrawal, S., & Silkari, S. (2015). Recent Advancement in Machine Learning Based Internet Traffic Classification. *Procedia Computer Science*, 60, 784–791. doi:10.1016/j.procs.2015.08.238
- Nazarenko, E., Varkentin, V., & Polyakova, T. (2019). Features of Application of Machine Learning Methods for Classification of Network Traffic (Features, Advantages, Disadvantages). *International Multi-Conference on Industrial Engineering and Modern Technologies (FarEastCon)* (pp. 1-5). Vladivostok: IEEE. doi:10.1109/FarEastCon.2019.8934236
- Nguyen, H. M., Cooper, E. W., & Kamei, K. (2011). Borderline over-sampling for imbalanced data classification. *International Journal of Knowledge Engineering and Soft Data Paradigms*, 3(1), 4-21. doi:10.1504/IJKESDP.2011.039875
- Nguyen, T. T., & Armitage, G. (2008). A survey of techniques for internet traffic classification using machine learning. *IEEE Communications Surveys & Tutorials*, 10(4), 56-76. doi:10.1109/SURV.2008.080406

- Patel, S., Gupta, A., Nikhil, Kumari, S., Singh, M., & Sharma, V. (2019). Network Traffic Classification Analysis Using Machine Learning Algorithms. *2018 International Conference on Advances in Computing, Communication Control and Networking (ICACCCN)* (pp. 1182-1187). Greater Noida: IEEE. doi:10.1109/ICACCCN.2018.8748290
- Peng, L., Yang, B., Chen, Y., & Abraham, A. (2009). Data gravitation based classification. *Information Science*, 179(6), 809-819. doi:10.1016/j.ins.2008.11.007
- Peng, L., Zhang, H., Chen, Y., & Yang, Y. B. (2017). Imbalanced traffic identification using an imbalanced data gravitation-based classification model. *Computer Communications*, 102, 177-189. doi:10.1016/j.comcom.2016.05.010
- Prashanth, G., Prashanth, V., Jayashree, P., & Srinivasan, N. (2008). Using Random Forests for Network-based Anomaly detection at Active routers. *2008 International Conference on Signal Processing, Communications and Networking*, pp. 93-96. doi:10.1109/ICSCN.2008.4447167
- Priya, A., Nandi, S., & Goswami, R. S. (2018). An Analysis of real-time network traffic for identification of browser and application of user using clustering algorithm. *2018 International Conference on Advances in Computing, Communication Control and Networking (ICACCCN)* (pp. 441-445). Greater Noida: IEEE. doi:10.1109/ICACCCN.2018.8748706
- Rizzi, A., Colabrese, S., & Baiocchi, A. (2013). Low complexity, high performance neuro-fuzzy system for Internet traffic flows early classification. *2013 9th International Wireless Communications and Mobile Computing Conference (IWCMC)* (pp. 77-82). Sardinia: IEEE. doi:10.1109/IWCMC.2013.6583538
- Rojas, J. S. (2017). *Kaggle*. (Kaggle) Retrieved January 2020, from <https://www.kaggle.com>
- Sáez, J. A., Krawczyk, B., & Woźniak, M. (2016). Analyzing the oversampling of different classes and types of examples in multi-class imbalanced datasets. *Pattern Recognition*, 57, 164-178. doi:10.1016/j.patcog.2016.03.012
- Seiffert C., Khoshgoftaar T. M., Van Hulse J., & A., N. (2010). RUSBoost: A Hybrid Approach to Alleviating Class Imbalance. *IEEE Transactions on Systems, Man, and Cybernetics Part A: Systems and Humans*, 40(1), 185-197. doi:10.1109/tsmca.2009
- Sen, S., Spatscheck, O., & Wang, D. (2004). Accurate, scalable in-network identification of P2P traffic using application signatures. *Proceedings of the 13th International Conference on World Wide Web* (pp. 512-521). ACM. doi:10.1145/988672.988742

- Singh, A., Thakur, N., & Sharma, A. (2016). A review of supervised machine learning algorithms. *2016 3rd International Conference on Computing for Sustainable Global Development (INDIACom)*, pp. 1310-1315.
- Soysal, M., & Schmidt, E. G. (2010). Machine learning algorithms for accurate flow-based network traffic classification: Evaluation and comparison. *Performance Evaluation*, 67(6), 451-467. doi:10.1016/j.peva.2010.01.001
- Sun, Z., Song, Q., Zhu, X., Sun, H., Xu, B., & Zhou, Y. (2015). A novel ensemble method for classifying imbalanced data. *Pattern Recognition*, 48(5), 1623-1637. doi:10.1016/j.patcog.2014.11.014
- Thai-Nghe, N., Gantner, Z., & Schmidt-Thieme, L. (2010). Cost-sensitive learning methods for imbalanced data. *The 2010 International Joint Conference on Neural Networks (IJCNN)*, (pp. 1-8). Barcelona. doi:10.1109/IJCNN.2010.5596486
- Theofilatos, A., Chen, C., & Antoniou, C. (2019). Comparing Machine Learning and Deep Learning Methods for Real-Time Crash Prediction. *Transportation Research Record*, 2673(8), 169-178. doi:10.1177/0361198119841571
- Tomek, I. (1976). Two Modifications of CNN. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-6(11), 769-772. doi:10.1109/TSMC.1976.4309452
- Tripathi, A., Goswami, T., Trivedi, S. K., & Sharma, R. D. (2021). A multi class random forest (MCRF) model for classification of small plant peptides. *International Journal of Information Management Data Insights*, 1(2). doi:10.1016/j.jjime.2021.100029
- Vu, L., Tra, D. V., & Nguyen, Q. U. (2016). Learning from Imbalanced data for encrypted traffic identification problem. *Proc. Of the Seventh Symposium on Information and Communication Technology*, (pp. 147-152). Ho Chi Minh. doi:10.1145/3011077.3011132
- Wang, Q., Yahyavi, A., Kemme, B., & He, W. (2015). I know what you did on your smartphone: Inferring app usage over encrypted data traffic. *2015 IEEE Conference on Communications and Network Security (CNS)* (pp. 433-441). Florence: IEEE. doi:10.1109/CNS.2015.7346855
- Wilson, D. L. (1972). Asymptotic Properties of Nearest Neighbor Rules Using Edited Data. *IEEE Trans. Syst. Man Cybern.*, 2(3), 408-421.
- Xiao, X., Li, R., Zheng, H.-T., Ye, R., Sangaiah, A. K., & Xia, S. (2019). Novel dynamic multiple classification system for network traffic. *Information Sciences*, 479, 526-541. doi:10.1016/j.ins.2018.10.039

- Zhang, H., Lu, G., Qassrawi, M. T., Zhang, Y., & Yu, X. (2012). Feature Selection for Optimizing Traffic Classification. *Computer Communications*, 35(12), 1457-1471. doi:10.1016/j.comcom.2012.04.012
- Zhang, J., & Mani, I. (2003). KNN Approach to Unbalanced Data Distributions: A Case Study Involving Information Extraction. *Proceeding of International Conference on Machine Learning (ICML 2003), Workshop on Learning from Imbalanced Data Sets*, (pp. 2-6). Washington DC.
- Zhen, L., & Qiong, L. (2012). Balanced feature selection method for internet traffic classification. *IET Network*, 1(2), 74–83. doi:10.1049/iet-net.2011.0049
- Zhou, Y.-F. L.-H. (2010, July). Cost-Sensitive Semi-Supervised Support Vector Machine. *Proceedings of the National Conference on Artificial Intelligence*, 24(1), pp. 500-505.